
Distributed Real-Time Estimation and Control for Heterogeneous UAV Clusters

A Unified Framework for Quaternion-Modelled Hexacopters
and Fixed-Wing Aircraft with Constrained
Moving-Horizon Estimation and Model-Predictive Control

Technical Report

Complete first-principles derivations of a hexacopter model with full quaternion attitude dynamics and of a six-degree-of-freedom fixed-wing model, together with a distributed moving-horizon estimation (DMHE) and distributed model-predictive control (DMPC) architecture built around a real-time constrained nonlinear MPC controller and a real-time constrained moving-horizon estimator running as the per-agent computational core.

Sreeram Anil

Public edition — the per-agent solver algorithms are separate ongoing research and are specified here by interface and measured performance only (see the scope note, Ch. 5).

Friedrich–Alexander–Universität Erlangen–Nürnberg

A reference C / C++ / Python implementation accompanies this report.
All model equations and numerical results are reproducible from the accompanying code.

September 20, 2026

Abstract

This report develops, from first principles, a complete mathematical and software framework for the distributed real-time estimation and control of clusters of unmanned aerial vehicles (UAVs) comprising two dynamically dissimilar vehicle classes: multirotor *hexacopters*, modelled with full unit-quaternion attitude dynamics, and *fixed-wing* aircraft, modelled with the standard six-degree-of-freedom rigid-body aerodynamics of Beard and McLain instantiated on the Aerosonde airframe as a small-scale surrogate for a medium-altitude long-endurance platform.

The two vehicle models are derived in full: every reference frame, kinematic relation, force and moment, and Jacobian is stated explicitly, and every symbol is defined at the point of use. For the hexacopter we derive the Hamilton quaternion kinematics $\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes [0, \boldsymbol{\omega}]$, the Newton–Euler translational and rotational dynamics, the rotor thrust/torque model, and the control-allocation mixer that maps six rotor speeds to the collective-thrust/body-torque wrench, including its rank structure and single-rotor-failure redundancy. For the fixed-wing aircraft we derive the wind triangle, the sigmoid-blended stall aerodynamics, the full force and moment coefficient expansions, and the coupled rotational dynamics in the product-of-inertia Γ form, together with a numerically computed straight-and-level trim.

Each agent runs two constrained real-time-iteration solvers as its onboard computational core: a nonlinear model-predictive *controller* and a moving-horizon *estimator*, both of which impose general nonlinear state and input constraints on the full problem and execute in the real-time-iteration pattern, one iteration per sample, with the measurement-to-input path reduced to a single affine evaluation. Around this core we construct a distributed moving-horizon estimation (DMHE) layer, in which agents fuse relative inter-agent measurements to remain localised through GNSS outages, and a distributed model-predictive control (DMPC) layer that couples the agents through formation objectives and control-barrier-function collision-avoidance constraints, coordinated over a communication graph by consensus and alternating-direction methods.

A dependency-free reference implementation—C for the numerical core, C++ for the coordination layer, and Python for simulation—accompanies the report. In closed-loop simulation on the full nonlinear plants, a six-hexacopter cluster assembles and reconfigures a formation while maintaining collision-free separation, and the DMHE keeps GNSS-denied agents localised to within 9 cm of ground truth through a ten-second blackout during which a dead-reckoning filter drifts past 1.2 m; a four-aircraft fixed-wing cluster holds a coordinated-turn echelon formation. A 126-run Monte-Carlo campaign—over noise realisations, GNSS-denial severities from one to five of six agents, degraded communication graphs and an estimator ablation—records zero divergences and zero solver failures, a 95th-percentile denied-agent error below 0.12 m whenever every denied agent retains a localised neighbour, and isolates *anchor coverage* of the communication graph as the condition on which that resilience rests. A computational study instantiates the embedded solvers on the drone models themselves: one constrained real-time-iteration cycle of the full quaternion-hexacopter NMPC executes in about a hundred microseconds re-linearised, or below ten microseconds median in its warm-started mode, with a 30 ns measurement-to-input latency and accuracy identical across modes. The embedded implementations are finally substi-

tuted into the closed loop itself: over paired-seed cluster campaigns the deployed C estimator and controller reproduce the reference behaviour with zero safety violations while *improving* the estimation error to six centimetres, and a leveled multi-hop anchoring extension of the DMHE restores decimetre accuracy in the anchor-coverage-broken cases, so the complete distributed estimation-and-control loop is validated with the embedded solvers in it on the nonlinear multi-agent plant. Every reported number is produced by the accompanying code.

Contents

Abstract	i
Nomenclature	vi
1 Introduction	1
1.1 Motivation	1
1.2 Problem statement	1
1.3 Approach and contributions	2
1.4 Report organisation	3
2 Mathematical Preliminaries and Notation	4
2.1 Reference frames	4
2.2 Rotations and the group $\text{SO}(3)$	4
2.3 Unit quaternions	5
2.4 Rigid-body dynamics	6
2.5 Algebraic graph theory for the cluster	6
3 Hexacopter Dynamic Model with Quaternion Attitude	8
3.1 Frames, state and inputs	8
3.2 Attitude kinematics	8
3.3 Translational dynamics	9
3.4 Rotational dynamics	9
3.5 Rotor thrust and torque	9
3.6 Control allocation (the mixer)	10
3.7 Assembled nonlinear model	11
3.8 Discretisation and Jacobians	11
3.9 Differential flatness	11
3.10 Numerical validation	12
4 Fixed-Wing UAV Dynamic Model	13
4.1 Frames, state and controls	13
4.2 Airspeed, angle of attack and sideslip	13
4.3 Kinematics	14

4.4	Forces and moments	14
4.5	Rigid-body dynamics	15
4.6	Trim and validation	16
5	Per-Agent Real-Time Estimation and Control	17
5.1	The model-predictive controller	17
5.2	Real-time iteration	18
5.3	The moving-horizon estimator	18
5.4	The C application programming interface	19
6	Distributed Moving-Horizon Estimation	21
6.1	Cluster measurement model	21
6.2	Local estimator and the translational reduction	21
6.3	Relative fusion and the data-incest problem	22
6.4	Stability and what is claimed	22
7	Distributed Model-Predictive Control	24
7.1	Cluster objective and coupling	24
7.2	Consensus on the formation reference	24
7.3	Coupling constraints by decomposition	25
7.4	Real-time safety filter	25
8	Formation, Clustering and Coordination	27
8.1	Formation specifications	27
8.2	The communication graph and its maintenance	27
8.3	Reconfiguration and fault handling	28
9	Software Architecture	29
9.1	Layered design	29
9.2	Autopilot integration	30
9.3	Why the core is preserved, not rewritten	30
10	Simulation Study	31
10.1	Hexacopter cluster: representative mission	31
10.2	Monte-Carlo robustness study	32
10.3	Computational study of the embedded per-agent solvers	34
10.4	The embedded solvers in the loop	35
10.5	Fixed-wing cluster	37
10.6	Discussion	38
11	Conclusions and Future Work	40
A	Quaternion Identities and Proofs	42

B	Hexacoiter Jacobians	44
C	Vehicle Parameter Tables	46
C.1	Hexacoiter	46
C.2	Fixed-wing (Aerosonde)	46

Nomenclature

The following conventions hold throughout unless stated otherwise. Scalars are italic (m , T), vectors bold lower case ($\mathbf{p}$, $\boldsymbol{\omega}$), matrices bold upper case ($\mathbf{R}$, $\mathbf{J}$), and quaternions bold ($\mathbf{q}$). A superscript on a vector denotes the frame in which it is resolved, *e.g.* $\mathbf{v}^b$ is $\mathbf{v}$ expressed in the body frame. All symbols are additionally defined at first use.

Frames and operators

$\mathcal{F}^i, \mathcal{F}^b, \mathcal{F}^s, \mathcal{F}^w$	inertial (world), body, stability and wind frames
$\mathbf{R} \in \text{SO}(3)$	rotation matrix; $\mathbf{R}_b^i$ maps body to inertial
$\text{SO}(3), \mathfrak{so}(3)$	special orthogonal group and its Lie algebra (skew matrices)
$[\mathbf{a}]_{\times}$	skew-symmetric matrix with $[\mathbf{a}]_{\times} \mathbf{b} = \mathbf{a} \times \mathbf{b}$
$\mathbf{q} = (q_w, \mathbf{q}_v)$	unit quaternion, scalar-first, Hamilton convention
$\otimes$	Hamilton quaternion product
$\mathbf{q}^*$	quaternion conjugate; $(\cdot)^\top$ transpose; $\ \cdot\ $ Euclidean norm
Exp, Log	exponential / logarithm maps on $\text{SO}(3)$ and $\mathbb{S}^3$

Rigid-body state (both vehicle classes)

$\mathbf{p} = (p_n, p_e, p_d)$	inertial position (m); NED for fixed-wing, ENU-up for multirotor
$\mathbf{v}$	inertial (multirotor) or body-frame (fixed-wing) velocity (m/s)
$\boldsymbol{\omega} = (p, q, r)$	body angular velocity (rad/s)
$m, \mathbf{J}$	mass (kg), inertia tensor (kg m^2)
g	gravitational acceleration, 9.81 m/s^2
ϕ, θ, ψ	roll, pitch, yaw Euler angles (3–2–1)

Hexacopter-specific

$T, \boldsymbol{\tau}$	collective thrust (N) and body torque (N m)
Ω_i, f_i	speed and thrust of rotor i ; $f_i = k_f \Omega_i^2$
k_f, k_m	rotor thrust and drag-torque coefficients
L, β_i, σ_i	arm length, rotor azimuth, spin direction ± 1
$\mathbf{M}$	4×6 control-allocation (mixer) matrix

Fixed-wing-specific

V_a, α, β	airspeed (m/s), angle of attack, sideslip (rad)
ρ, S, b, c	air density, wing area, span, mean chord
C_L, C_D, C_m	lift, drag, pitch-moment coefficients
C_Y, C_ℓ, C_n	side-force, roll- and yaw-moment coefficients
$\delta_e, \delta_a, \delta_r, \delta_t$	elevator, aileron, rudder, throttle

Estimation, control and coordination

N, M	MPC prediction horizon, MHE estimation window
ℓ_k, ℓ_N	MPC stage and terminal cost
$c(\cdot) \leq 0, c_N(\cdot) \leq 0$	stage and terminal inequality constraints
λ, μ	equality (co-state) and inequality multipliers
w	estimated disturbance input (MHE); Λ_a, x_a arrival information/mean
$\mathcal{G} = (\mathcal{V}, \mathcal{E})$	communication graph; $\mathcal{N}_i$ neighbours of i
$\mathbf{L}, \lambda_2(\mathbf{L})$	graph Laplacian and algebraic connectivity
$h(\cdot), \alpha(\cdot)$	control-barrier function and extended class- $\mathcal{K}$ function
d_i	desired formation offset of agent i

Chapter 1

Introduction

1.1 Motivation

Autonomous aerial systems are moving from single, monolithic vehicles toward *clusters*: coordinated groups of many UAVs that share sensing, computation and objectives. A cluster survives the loss of individual members, covers more ground per unit time, localises targets by triangulation that no single vehicle can achieve, and reshapes itself on demand into search lines, perimeter rings or convoy escorts. Realising these behaviours in the field forces three problems together that are usually studied in isolation.

First, the vehicles are *dynamically heterogeneous*. A multirotor hexacopter hovers, translates in any direction and reorients almost instantaneously, but is energetically expensive and slow; a fixed-wing aircraft is efficient and fast but cannot stop, must maintain airspeed above stall and turns only by banking. A framework that is to command a mixed fleet must model both to the same fidelity and expose a common control interface. Second, the vehicles operate under *hard physical constraints*—thrust and actuator limits, minimum airspeeds, current and voltage envelopes, no-fly volumes and, above all, mutual collision avoidance—that are safety-critical and active exactly when the mission is most demanding. Third, coordination must be *distributed and real-time*: bandwidth is limited, a central node is a single point of failure, and every vehicle must close its own estimation and control loops at rates measured in hundreds of hertz on an embedded companion computer.

This report addresses the three problems jointly. It develops complete, first-principles dynamic models of a quaternion-attitude hexacopter and of a six-degree-of-freedom fixed-wing aircraft; it places at the core of each agent a constrained real-time nonlinear model-predictive *controller* and a constrained real-time moving-horizon *estimator*; and it builds around that core a distributed estimation-and-control architecture that couples the agents over a communication graph into safe, coordinated formations.

1.2 Problem statement

Consider a cluster of N_a UAVs indexed by $i \in \mathcal{V} = \{1, \dots, N_a\}$, each with nonlinear continuous-time dynamics

$$\dot{\mathbf{x}}_i = \mathbf{f}_i(\mathbf{x}_i, \mathbf{u}_i), \quad \mathbf{y}_i = \mathbf{h}_i(\mathbf{x}_i) + \mathbf{n}_i, \quad (1.1)$$

where $\mathbf{x}_i$ is the vehicle state, $\mathbf{u}_i$ its control input, $\mathbf{y}_i$ a vector of noisy onboard and inter-agent measurements, and $\mathbf{n}_i$ measurement noise. The vehicle class of agent i —hexacopter or fixed-wing—fixes the form of $\mathbf{f}_i$ and $\mathbf{h}_i$, derived in [Chapters 3](#) and [4](#). The agents share a time-varying communication graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$; agent i may exchange messages only with its neighbours

$\mathcal{N}_i = \{j : (i, j) \in \mathcal{E}\}$. Each agent must, using only local measurements and neighbour messages,

1. *estimate* its own state $\mathbf{x}_i$ (and unmeasured disturbances) in real time, remaining accurate even when a subset of its absolute-positioning measurements is lost;
2. *control* its motion to track a share of a cluster-level objective (a formation, a trajectory, a coverage pattern) while respecting its own actuator and state constraints; and
3. *coordinate* with neighbours so that the cluster as a whole is collision-free, connected and consistent.

The design target throughout is a per-agent computation and communication budget small enough to run on an embedded companion computer at the vehicle’s control rate.

1.3 Approach and contributions

The architecture is layered, and the layers map onto the natural language boundaries of an implementation (Chapter 9):

- a *numerical core*, in C, providing the two constrained real-time solvers—the MPC controller and the MHE estimator—as the per-agent engine (Chapter 5);
- a *coordination layer*, in C++, implementing the distributed estimation (Chapter 6) and distributed control (Chapters 7 and 8) around that core, together with neighbour discovery, consensus, message passing and fault handling; and
- a *research layer*, in Python/MATLAB, for simulation, tuning and analysis (Chapter 10).

The specific contributions are the following.

1. **A complete quaternion hexacopter model (Chapter 3)**, derived from first principles: Hamilton quaternion kinematics, Newton–Euler translational and rotational dynamics, the rotor thrust/torque model, and the six-rotor control allocation with its rank structure, invertibility and fault tolerance, followed by discretisation, analytic Jacobians and a differential-flatness map. All relations are validated numerically.
2. **A complete fixed-wing model (Chapter 4)** in the Beard–McLain six-degree-of-freedom formulation, including the wind triangle, the sigmoid-blended stall aerodynamics, the full force and moment coefficient expansions, the product-of-inertia rotational dynamics, and a numerically solved trim, on the fully parameterised Aerosonde airframe.
3. **A generic account of the per-agent constrained real-time estimator and controller (Chapter 5)** as a matched estimation–control pair that imposes general nonlinear state and input constraints on the full problem and executes one real-time iteration per sample, exposed to the coordination layer through a clean C application programming interface. The solvers’ internal algorithms are separate unpublished work and are treated here as a fixed, validated capability specified by that interface (see the scope note in Chapter 5).
4. **A distributed moving-horizon estimator (Chapter 6)** in which agents fuse relative inter-agent measurements to their well-localised neighbours, with a covariance-aware fusion rule that keeps GNSS-denied agents accurate through absolute-positioning outages.
5. **A distributed model-predictive controller (Chapters 7 and 8)** that couples the agents through formation objectives and control-barrier-function collision-avoidance constraints, coordinated by consensus and alternating-direction methods, with the cooperative-DMPC stability and recursive-feasibility properties stated and attributed.
6. **A reference implementation and simulation study (Chapters 9 and 10)** on the full nonlinear plants, exercising formation assembly, reconfiguration, collision avoidance and GNSS-denied localisation for both vehicle classes.

1.4 Report organisation

[Chapter 2](#) fixes the frames, rotation representations, rigid-body mechanics and graph-theoretic tools used throughout. [Chapters 3](#) and [4](#) derive the two vehicle models. [Chapter 5](#) describes the per-agent estimator and controller. [Chapters 6](#) to [8](#) build the distributed estimation, control and formation layers. [Chapter 9](#) sets out the software architecture and [Chapter 10](#) the simulation study. [Chapter 11](#) concludes. Appendices collect quaternion identities, the full hexacopter Jacobians and the complete vehicle parameter tables.

A note on the per-agent solvers. Throughout this report the two onboard solvers are referred to generically as “the MPC controller” and “the MHE estimator”. Their internal algorithms are treated as a fixed, validated capability accessed through the interface of [Chapter 5](#); the distributed framework depends only on that interface and not on the solvers’ internal construction.

Chapter 2

Mathematical Preliminaries and Notation

This chapter fixes the reference frames, the representations of rotation, the general rigid-body equations of motion and the graph-theoretic tools used in the rest of the report. Every object introduced here reappears in the vehicle models of [Chapters 3 and 4](#) and in the distributed layers of [Chapters 6 to 8](#).

2.1 Reference frames

A *frame* is a right-handed orthonormal triad fixed to a body or to the environment. Four frames are used.

- The **inertial frame** $\mathcal{F}^i$ is fixed to the ground and treated as non-accelerating. For the fixed-wing aircraft it is a North–East–Down (NED) frame, with $\mathbf{e}_3$ pointing toward the centre of the Earth; for the multirotor we use a local East–North–Up (ENU), z -up frame, because thrust then acts along $+\mathbf{e}_3$. The choice is purely one of convenience and is stated with each model.
- The **body frame** $\mathcal{F}^b$ is fixed to the vehicle at its centre of mass, with $\mathbf{b}_1$ forward, $\mathbf{b}_2$ to the right (NED) or left (ENU) and $\mathbf{b}_3$ completing the triad.
- The **stability** and **wind** frames $\mathcal{F}^s, \mathcal{F}^w$, used only for the fixed-wing aircraft, are obtained from $\mathcal{F}^b$ by rotations through the angle of attack α and the sideslip angle β ([Section 4.2](#)).

A vector $\mathbf{a}$ resolved in frame $\mathcal{F}^x$ is written $\mathbf{a}^x$. [Figure 2.1](#) shows the inertial and body frames.

2.2 Rotations and the group $\text{SO}(3)$

A rotation is an element of the *special orthogonal group*

$$\text{SO}(3) = \{\mathbf{R} \in \mathbb{R}^{3 \times 3} : \mathbf{R}^\top \mathbf{R} = \mathbf{I}_3, \det \mathbf{R} = +1\}. \quad (2.1)$$

The matrix $\mathbf{R}_b^i$ maps body-resolved coordinates to inertial coordinates, $\mathbf{a}^i = \mathbf{R}_b^i \mathbf{a}^b$. Its columns are the body axes expressed in $\mathcal{F}^i$. Because $\mathbf{R}$ is orthogonal, $(\mathbf{R}_b^i)^{-1} = \mathbf{R}_b^{i\top} = \mathbf{R}_i^b$.

The hat map. For $\mathbf{a} = (a_1, a_2, a_3)^\top \in \mathbb{R}^3$ define the skew-symmetric *hat* matrix

$$[\mathbf{a}]_\times = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix} \in \mathfrak{so}(3), \quad [\mathbf{a}]_\times \mathbf{b} = \mathbf{a} \times \mathbf{b}. \quad (2.2)$$

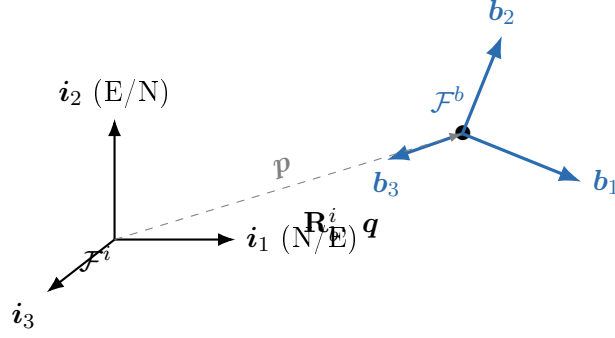


Figure 2.1: Inertial frame $\mathcal{F}^i$ and body frame $\mathcal{F}^b$. The vehicle position $\mathbf{p}$ is resolved in $\mathcal{F}^i$; the attitude is the rotation $\mathbf{R}_b^i \in \text{SO}(3)$, equivalently the unit quaternion $\mathbf{q}$, that maps body-resolved vectors into the inertial frame.

The set $\mathfrak{so}(3)$ of skew matrices is the Lie algebra of $\text{SO}(3)$; it is the tangent space at the identity. The kinematics of a rotating rigid body live in this algebra: if $\boldsymbol{\omega}^b$ is the body angular velocity, then

$$\dot{\mathbf{R}}_b^i = \mathbf{R}_b^i [\boldsymbol{\omega}^b]_{\times}. \quad (2.3)$$

Equation (2.3) is the rotation-matrix attitude kinematics; the quaternion form derived next is equivalent but uses four parameters instead of nine and integrates without loss of orthogonality.

Euler angles. The 3–2–1 (yaw–pitch–roll) Euler angles (ϕ, θ, ψ) parameterise $\mathbf{R}_b^i$ as the product $\mathbf{R}_z(\psi)\mathbf{R}_y(\theta)\mathbf{R}_x(\phi)$. They are intuitive and used for the fixed-wing aircraft, but are singular at $\theta = \pm\frac{\pi}{2}$ (gimbal lock); the multirotor, which may pitch or roll to any angle, therefore uses quaternions.

2.3 Unit quaternions

A *quaternion* is a four-tuple $\mathbf{q} = (q_w, q_x, q_y, q_z)^\top = (q_w, \mathbf{q}_v)^\top$ with scalar part $q_w \in \mathbb{R}$ and vector part $\mathbf{q}_v \in \mathbb{R}^3$. We use the **Hamilton** convention throughout (the convention in which $ij = k$). The quaternion (Hamilton) product of $\mathbf{p} = (p_w, \mathbf{p}_v)$ and $\mathbf{q} = (q_w, \mathbf{q}_v)$ is

$$\mathbf{p} \otimes \mathbf{q} = \begin{bmatrix} p_w q_w - \mathbf{p}_v^\top \mathbf{q}_v \\ p_w \mathbf{q}_v + q_w \mathbf{p}_v + \mathbf{p}_v \times \mathbf{q}_v \end{bmatrix}. \quad (2.4)$$

The product is associative and bilinear but *not* commutative. The *conjugate* is $\mathbf{q}^* = (q_w, -\mathbf{q}_v)$ and the norm is $\|\mathbf{q}\| = \sqrt{q_w^2 + \mathbf{q}_v^\top \mathbf{q}_v}$. A *unit quaternion*, $\|\mathbf{q}\| = 1$, represents a rotation; the set of unit quaternions is the three-sphere $\mathbb{S}^3$, and $\mathbf{q}^* = \mathbf{q}^{-1}$ for unit quaternions.

Rotation by a quaternion. A vector $\mathbf{a} \in \mathbb{R}^3$ is rotated by embedding it as the pure quaternion $(0, \mathbf{a})$ and conjugating,

$$(0, \mathbf{a}^i) = \mathbf{q} \otimes (0, \mathbf{a}^b) \otimes \mathbf{q}^*, \quad (2.5)$$

which is realised by the rotation matrix

$$\mathbf{R}(\mathbf{q}) = (q_w^2 - \mathbf{q}_v^\top \mathbf{q}_v) \mathbf{I}_3 + 2 \mathbf{q}_v \mathbf{q}_v^\top + 2 q_w [\mathbf{q}_v]_{\times}. \quad (2.6)$$

Writing $\mathbf{q} = (q_w, q_x, q_y, q_z)$, Eq. (2.6) expands to

$$\mathbf{R}(\mathbf{q}) = \begin{bmatrix} 1 - 2(q_y^2 + q_z^2) & 2(q_x q_y - q_w q_z) & 2(q_x q_z + q_w q_y) \\ 2(q_x q_y + q_w q_z) & 1 - 2(q_x^2 + q_z^2) & 2(q_y q_z - q_w q_x) \\ 2(q_x q_z - q_w q_y) & 2(q_y q_z + q_w q_x) & 1 - 2(q_x^2 + q_y^2) \end{bmatrix}. \quad (2.7)$$

The map $\mathbf{q} \mapsto \mathbf{R}(\mathbf{q})$ is a two-to-one homomorphism: $\mathbf{q}$ and $-\mathbf{q}$ give the same rotation, and $\mathbf{R}(\mathbf{p} \otimes \mathbf{q}) = \mathbf{R}(\mathbf{p})\mathbf{R}(\mathbf{q})$. These identities are proved and verified in [Appendix A](#).

Quaternion kinematics. If the body rotates with angular velocity $\boldsymbol{\omega}^b$, the attitude quaternion evolves by

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes (0, \boldsymbol{\omega}^b) = \frac{1}{2} \boldsymbol{\Omega}(\boldsymbol{\omega}^b) \mathbf{q} \quad (2.8)$$

where, using [Eq. \(2.4\)](#), the right-multiplication is written as the linear map

$$\boldsymbol{\Omega}(\boldsymbol{\omega}) = \begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix}. \quad (2.9)$$

Equation (2.8) is the central kinematic relation used in the hexacopter model; it is derived in [Appendix A](#) and is the quaternion counterpart of [Eq. \(2.3\)](#).

Exponential map and norm-preserving integration. The exponential map $\text{Exp} : \mathbb{R}^3 \rightarrow \mathbb{S}^3$ takes a rotation vector $\boldsymbol{\theta} = \theta \mathbf{u}$ ($\|\mathbf{u}\| = 1$) to the unit quaternion

$$\text{Exp}(\boldsymbol{\theta}) = \left(\cos \frac{\theta}{2}, \mathbf{u} \sin \frac{\theta}{2} \right). \quad (2.10)$$

Holding $\boldsymbol{\omega}^b$ constant over a step of length Δt , the exact solution of [Eq. \(2.8\)](#) is the group update

$$\mathbf{q}_{k+1} = \mathbf{q}_k \otimes \text{Exp}(\boldsymbol{\omega}^b \Delta t), \quad (2.11)$$

which preserves $\|\mathbf{q}\| = 1$ to machine precision and is used by the integrator of [Chapter 3](#). This is a genuine advantage over [Eq. \(2.3\)](#): a Runge–Kutta step applied to the nine entries of $\mathbf{R}$ drifts off $\text{SO}(3)$ and must be re-orthonormalised.

2.4 Rigid-body dynamics

Both vehicle classes are rigid bodies, and both obey the Newton–Euler equations. Let m be the mass, $\mathbf{J} \in \mathbb{R}^{3 \times 3}$ the inertia tensor about the centre of mass (resolved in $\mathcal{F}^b$), $\mathbf{F}$ the resultant external force and $\boldsymbol{\tau}^b$ the resultant external moment about the centre of mass. Newton’s second law in the inertial frame and Euler’s rotational equation in the body frame read

$$m \dot{\mathbf{v}}^i = \mathbf{F}^i, \quad (2.12)$$

$$\mathbf{J} \dot{\boldsymbol{\omega}}^b = -\boldsymbol{\omega}^b \times (\mathbf{J} \boldsymbol{\omega}^b) + \boldsymbol{\tau}^b. \quad (2.13)$$

The gyroscopic term $-\boldsymbol{\omega} \times \mathbf{J} \boldsymbol{\omega}$ in [Eq. \(2.13\)](#) arises because Euler’s equation is written in the rotating body frame; it couples the three rotational axes whenever $\mathbf{J}$ is not isotropic. The two vehicle models differ only in how $\mathbf{F}$ and $\boldsymbol{\tau}$ are generated—by rotors ([Chapter 3](#)) or by aerodynamic surfaces and a propeller ([Chapter 4](#))—and in whether translation is written in the inertial or body frame.

2.5 Algebraic graph theory for the cluster

The communication topology of the cluster is a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with vertices $\mathcal{V} = \{1, \dots, N_a\}$ (the agents) and edges $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ (the links). The graph is *undirected* if $(i, j) \in \mathcal{E} \Leftrightarrow (j, i) \in \mathcal{E}$. The *neighbourhood* of agent i is $\mathcal{N}_i = \{j : (i, j) \in \mathcal{E}\}$, and its *degree* is $d_i = |\mathcal{N}_i|$.

Adjacency, degree and Laplacian. The adjacency matrix $\mathbf{A} \in \mathbb{R}^{N_a \times N_a}$ has $A_{ij} = 1$ if $(i, j) \in \mathcal{E}$ and 0 otherwise; the degree matrix is $\mathbf{D} = \text{diag}(d_1, \dots, d_{N_a})$; and the *graph Laplacian* is

$$\mathbf{L} = \mathbf{D} - \mathbf{A}. \quad (2.14)$$

The Laplacian is symmetric positive semidefinite with eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_{N_a}$. The all-ones vector $\mathbf{1}$ is always in its kernel, $\mathbf{L}\mathbf{1} = \mathbf{0}$. The second-smallest eigenvalue $\lambda_2(\mathbf{L})$, the *algebraic connectivity* or Fiedler value, is strictly positive if and only if the graph is connected, and it sets the convergence rate of the consensus and distributed-optimisation algorithms of [Chapters 6 and 7](#).

Consensus. The prototypical distributed operation is *average consensus*. If each agent holds a value z_i and updates it by

$$\dot{z}_i = - \sum_{j \in \mathcal{N}_i} (z_i - z_j), \quad \text{equivalently} \quad \dot{\mathbf{Z}} = -\mathbf{L}\mathbf{Z}, \quad (2.15)$$

then on a connected graph every z_i converges to the average $\frac{1}{N_a} \sum_j z_j(0)$ at a rate governed by $\lambda_2(\mathbf{L})$. In discrete time we use the *Metropolis–Hastings* weights

$$W_{ij} = \frac{1}{1 + \max(d_i, d_j)} \quad (j \in \mathcal{N}_i), \quad W_{ii} = 1 - \sum_{j \in \mathcal{N}_i} W_{ij}, \quad (2.16)$$

which make the update matrix $\mathbf{W}$ doubly stochastic with spectral radius on the disagreement subspace strictly below one, so the iteration $z_i^+ = \sum_j W_{ij} z_j$ is contractive for *any* connected graph and *any* degree. (A naive fixed step $z_i^+ = z_i + \epsilon \sum_{j \in \mathcal{N}_i} (z_j - z_i)$ is stable only for $\epsilon < 1/d_{\max}$; the Metropolis weights remove this tuning constraint, a fact we rely on in the coordination layer.) These tools are developed further, with references, in [Chapter 8](#).

Chapter 3

Hexacopter Dynamic Model with Quaternion Attitude

This chapter derives the complete nonlinear dynamic model of a hexacopter from first principles. The attitude is represented by a unit quaternion (Section 2.3), which avoids the gimbal-lock singularity of Euler angles and admits the norm-preserving integrator of Eq. (2.11)—both essential for a vehicle that may execute large, aggressive attitude changes. The derivation proceeds from the state definition through the kinematics, the translational and rotational dynamics, the rotor and control-allocation models, to the assembled state-space form, its discretisation, Jacobians and differential-flatness structure. Numerical validation of every relation closes the chapter.

3.1 Frames, state and inputs

We use a z -up inertial frame $\mathcal{F}^i$ (ENU) so that the collective rotor thrust acts along the positive body axis $\mathbf{b}_3$ and gravity along $-\mathbf{e}_3$. The vehicle is a rigid body of mass m with inertia tensor $\mathbf{J} = \text{diag}(J_{xx}, J_{yy}, J_{zz})$ about the centre of mass; for the symmetric airframe modelled here the products of inertia vanish. The 13-dimensional state collects inertial position, inertial velocity, the body-to-inertial attitude quaternion and the body angular velocity,

$$\mathbf{x} = (\mathbf{p}^\top, \mathbf{v}^\top, \mathbf{q}^\top, \boldsymbol{\omega}^\top)^\top \in \mathbb{R}^{13}, \quad \mathbf{p}, \mathbf{v} \in \mathbb{R}^3, \quad \mathbf{q} \in \mathbb{S}^3, \quad \boldsymbol{\omega} \in \mathbb{R}^3. \quad (3.1)$$

The control input is the *wrench* of collective thrust and body torque,

$$\mathbf{u} = (T, \boldsymbol{\tau}^\top)^\top = (T, \tau_x, \tau_y, \tau_z)^\top \in \mathbb{R}^4, \quad (3.2)$$

which is realised physically by the six rotor speeds through the control allocation of Section 3.6. Treating the wrench as the input keeps the rigid-body dynamics independent of the rotor count and geometry; the allocation is a static, invertible map inserted between the controller and the actuators.

3.2 Attitude kinematics

The attitude quaternion $\mathbf{q} = \mathbf{R}_b^i$ maps body-resolved vectors to the inertial frame through $\mathbf{R}(\mathbf{q})$ of Eq. (2.7). With body angular velocity $\boldsymbol{\omega}$, the quaternion obeys the kinematic relation Eq. (2.8),

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes (0, \boldsymbol{\omega}) = \frac{1}{2} \boldsymbol{\Omega}(\boldsymbol{\omega}) \mathbf{q}, \quad (3.3)$$

with $\boldsymbol{\Omega}$ from Eq. (2.9). In a numerical integrator the unit-norm constraint is maintained exactly by the exponential update Eq. (2.11); between full steps we renormalise $\mathbf{q} \leftarrow \mathbf{q} / \|\mathbf{q}\|$ as a safeguard against floating-point drift.

3.3 Translational dynamics

The only forces on the airframe are gravity and the collective rotor thrust (aerodynamic drag is included as a lumped term below). The thrust acts along the body axis $\mathbf{b}_3$ with magnitude T ; expressed in the inertial frame it is $\mathbf{R}(\mathbf{q}) T \mathbf{e}_3$. Newton's second law Eq. (2.12) gives

$$\dot{\mathbf{p}} = \mathbf{v}, \quad (3.4)$$

$$m \dot{\mathbf{v}} = -mg \mathbf{e}_3 + \mathbf{R}(\mathbf{q}) T \mathbf{e}_3 - \mathbf{D}, \quad (3.5)$$

where $\mathbf{e}_3 = (0, 0, 1)^\top$, $g = 9.81 \text{ m/s}^2$, and $\mathbf{D} = c_D \mathbf{v}$ is a first-order translational drag with lumped coefficient $c_D \geq 0$ (set to a small value; it may be replaced by a rotor-drag model without changing the structure). Dividing Eq. (3.5) by m ,

$$\dot{\mathbf{v}} = -g \mathbf{e}_3 + \frac{T}{m} \mathbf{R}(\mathbf{q}) \mathbf{e}_3 - \frac{c_D}{m} \mathbf{v}. \quad (3.6)$$

The third column of $\mathbf{R}(\mathbf{q})$ is the body- z axis in inertial coordinates; Eq. (3.6) therefore says that the thrust accelerates the vehicle along wherever “up” currently points, which is the essential coupling between attitude and translation that the controller must invert (Chapter 5).

3.4 Rotational dynamics

Euler's equation Eq. (2.13) with the body torque $\boldsymbol{\tau}$ from the rotors gives

$$\mathbf{J} \dot{\boldsymbol{\omega}} = -\boldsymbol{\omega} \times (\mathbf{J} \boldsymbol{\omega}) + \boldsymbol{\tau}, \quad i.e. \quad \dot{\boldsymbol{\omega}} = \mathbf{J}^{-1}(\boldsymbol{\tau} - \boldsymbol{\omega} \times \mathbf{J} \boldsymbol{\omega}). \quad (3.7)$$

With $\mathbf{J} = \text{diag}(J_{xx}, J_{yy}, J_{zz})$ the gyroscopic cross-coupling expands componentwise to

$$\dot{\omega}_x = \frac{\tau_x + (J_{yy} - J_{zz}) \omega_y \omega_z}{J_{xx}}, \quad \dot{\omega}_y = \frac{\tau_y + (J_{zz} - J_{xx}) \omega_z \omega_x}{J_{yy}}, \quad \dot{\omega}_z = \frac{\tau_z + (J_{xx} - J_{yy}) \omega_x \omega_y}{J_{zz}}. \quad (3.8)$$

A residual gyroscopic torque from the spinning rotors, $\boldsymbol{\tau}_{\text{gyro}} = -\boldsymbol{\omega} \times \sum_i I_r \Omega_i \mathbf{e}_3$ with rotor inertia I_r , is negligible in balanced hover (the signed rotor-speed sum is near zero) and is omitted from the nominal model; it can be added to Eq. (3.7) without altering the structure.

3.5 Rotor thrust and torque

Each rotor i spins at angular speed Ω_i and, by momentum theory in the operating regime, produces a thrust and a reaction (drag) torque quadratic in Ω_i ,

$$f_i = k_f \Omega_i^2, \quad q_i = k_m \Omega_i^2, \quad (3.9)$$

with thrust coefficient $k_f > 0$ and drag-torque coefficient $k_m > 0$. The reaction torque acts about the rotor axis, opposing its spin: a rotor spinning counter-clockwise (angular momentum along $+\mathbf{b}_3$) exerts a reaction torque on the airframe along $-\mathbf{b}_3$, and vice versa. We let $\sigma_i \in \{+1, -1\}$ denote the *sign of the yaw reaction torque* of rotor i (so $\sigma_i = -1$ for a counter-clockwise rotor), and introduce the torque-to-thrust ratio $c_\tau = k_m/k_f$, so that the reaction torque about $\mathbf{b}_3$ is $\sigma_i c_\tau f_i$.

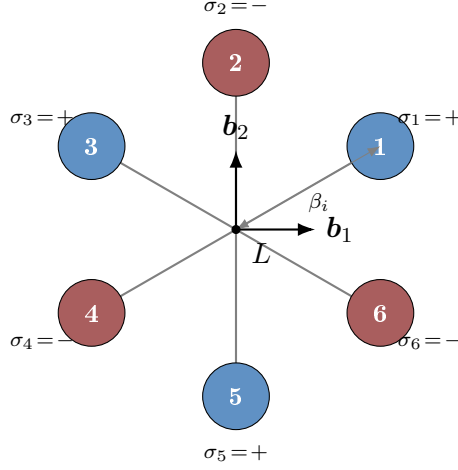


Figure 3.1: Hexacopter rotor geometry (top view, “X” configuration). Rotors are at azimuths β_i on arms of length L ; blue/red mark the two spin senses, whose yaw reaction-torque signs σ_i alternate around the ring so the hover reaction torque cancels ($\sum_i \sigma_i = 0$).

3.6 Control allocation (the mixer)

The six rotors sit at the ends of arms of length L at azimuth angles β_i around the body z -axis; for the “X” configuration used here, $\beta_i \in \{30^\circ, 90^\circ, 150^\circ, 210^\circ, 270^\circ, 330^\circ\}$, and the spin directions alternate, $\sigma_i = (+1, -1, +1, -1, +1, -1)$, so that the net reaction torque vanishes in hover (Fig. 3.1). Rotor i is located at $\mathbf{r}_i = L(\cos \beta_i, \sin \beta_i, 0)^\top$ and produces the body force $f_i \mathbf{e}_3$ and the reaction torque $\sigma_i c_\tau f_i \mathbf{e}_3$.

Summing the rotor contributions gives the wrench. The collective thrust is $T = \sum_i f_i$. The torque about the body axes is $\boldsymbol{\tau} = \sum_i (\mathbf{r}_i \times f_i \mathbf{e}_3 + \sigma_i c_\tau f_i \mathbf{e}_3)$; using $\mathbf{r}_i \times \mathbf{e}_3 = (r_{i,y}, -r_{i,x}, 0)^\top = L(\sin \beta_i, -\cos \beta_i, 0)^\top$,

$$\begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ L \sin \beta_1 & \cdots & & & L \sin \beta_6 \\ -L \cos \beta_1 & \cdots & & & -L \cos \beta_6 \\ \sigma_1 c_\tau & \cdots & & & \sigma_6 c_\tau \end{bmatrix}}_{\mathbf{M} \in \mathbb{R}^{4 \times 6}} \begin{bmatrix} f_1 \\ f_2 \\ \vdots \\ f_6 \end{bmatrix}, \quad f_i = k_f \Omega_i^2. \quad (3.10)$$

The 4×6 *mixer* $\mathbf{M}$ maps the six rotor thrusts to the four wrench components. Because there are six actuators and four controlled degrees of freedom, $\mathbf{M}$ has a two-dimensional null space: the hexacopter is *over-actuated*. This redundancy is the origin of its single-rotor-failure tolerance—with one rotor lost the remaining five still span the wrench space for the thrust and the two tilt torques—and is analysed in general in [1].

Given a commanded wrench $\mathbf{u}$, the rotor thrusts are recovered by the minimum-norm (Moore–Penrose) pseudo-inverse,

$$\mathbf{f}^* = \mathbf{M}^\dagger \mathbf{u}, \quad \mathbf{M}^\dagger = \mathbf{M}^\top (\mathbf{M} \mathbf{M}^\top)^{-1}, \quad \Omega_i = \sqrt{\max(f_i^*, 0)/k_f}, \quad (3.11)$$

followed by clipping to the physical speed range $0 \leq \Omega_i \leq \Omega_{\max}$. The pseudo-inverse selects, among all rotor-thrust vectors producing the demanded wrench, the one of least ℓ_2 norm, which minimises the electrical power to first order. Since $\mathbf{M}$ has full row rank (Proposition 3.1), $\mathbf{M} \mathbf{M}^\dagger = \mathbf{I}_4$, so the realised wrench equals the commanded wrench whenever the rotor speeds stay within their limits.

Proposition 3.1 (Mixer rank and hover). *For the “X” hexacopter with $L > 0$, $k_f > 0$, $c_\tau > 0$ and alternating spins, the mixer $\mathbf{M}$ in Eq. (3.10) has full row rank 4, and the hover wrench $\mathbf{u}_{\text{hov}} = (mg, 0, 0, 0)^\top$ is produced by the equal rotor speeds $\Omega_i = \sqrt{mg/(6k_f)}$ for all i .*

Sketch. The four rows of $\mathbf{M}$ are the constant row $\mathbf{1}^\top$, the two trigonometric rows $L \sin \beta_i$ and $-L \cos \beta_i$, and the alternating-sign row $\sigma_i c_\tau$. Over the six equally spaced azimuths these four length-six vectors are linearly independent, so $\text{rank} \mathbf{M} = 4$. At hover, equal thrusts $f_i = mg/6$ give $T = mg$; the trigonometric sums $\sum_i \sin \beta_i = \sum_i \cos \beta_i = 0$ annul τ_x, τ_y ; and the alternating signs sum to zero, annulling τ_z . Then $f_i = k_f \Omega_i^2$ gives the stated speed. The rank and the round-trip $\mathbf{M}\mathbf{M}^\dagger = \mathbf{I}_4$ are verified numerically in Section 3.10. $\square$

3.7 Assembled nonlinear model

Collecting Eqs. (3.3), (3.4), (3.6) and (3.7), the hexacopter is the 13-state, 4-input control-affine-in-thrust system

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \mathbf{v} \\ -g\mathbf{e}_3 + \frac{T}{m}\mathbf{R}(\mathbf{q})\mathbf{e}_3 - \frac{c_D}{m}\mathbf{v} \\ \frac{1}{2}\boldsymbol{\Omega}(\boldsymbol{\omega})\mathbf{q} \\ \mathbf{J}^{-1}(\boldsymbol{\tau} - \boldsymbol{\omega} \times \mathbf{J}\boldsymbol{\omega}) \end{bmatrix}. \quad (3.12)$$

This $\mathbf{f}$ is the vehicle dynamics $\mathbf{f}_i$ of Eq. (1.1) for a multirotor agent; it feeds directly into the MPC controller and MHE estimator of Chapter 5.

3.8 Discretisation and Jacobians

The solvers of Chapter 5 require a discrete-time map and its sensitivities. We integrate Eq. (3.12) over a sampling interval T_s with a fourth-order Runge–Kutta (RK4) scheme applied to the vector part $(\mathbf{p}, \mathbf{v}, \boldsymbol{\omega})$, while the quaternion is advanced by the exact group update Eq. (2.11) with the mean body rate over the step; this hybrid integrator keeps $\|\mathbf{q}\| = 1$ while retaining fourth-order accuracy in the translational and rotational states. The result is the discrete map

$$\mathbf{x}_{k+1} = \mathbf{f}_d(\mathbf{x}_k, \mathbf{u}_k), \quad \mathbf{A}_k = \left. \frac{\partial \mathbf{f}_d}{\partial \mathbf{x}} \right|_{\mathbf{x}_k, \mathbf{u}_k}, \quad \mathbf{B}_k = \left. \frac{\partial \mathbf{f}_d}{\partial \mathbf{u}} \right|_{\mathbf{x}_k, \mathbf{u}_k}. \quad (3.13)$$

The blocks of $\mathbf{A}_k$ and $\mathbf{B}_k$ follow by the chain rule through the RK4 stages; the analytic expressions for the continuous Jacobians $\partial \mathbf{f} / \partial \mathbf{x}$ and $\partial \mathbf{f} / \partial \mathbf{u}$ are collected in Appendix B. Two entries are worth noting because the controller relies on them: $\partial \dot{\mathbf{v}} / \partial T = \frac{1}{m}\mathbf{R}(\mathbf{q})\mathbf{e}_3$, the thrust axis, and $\partial \dot{\boldsymbol{\omega}} / \partial \boldsymbol{\tau} = \mathbf{J}^{-1}$, the inverse inertia.

3.9 Differential flatness

The hexacopter is *differentially flat* in the outputs $(\mathbf{p}, \psi)$ —position and yaw [2]. This structure is used by the position controller of Chapter 7 to convert a commanded inertial acceleration $\mathbf{a}_{\text{cmd}}$ into a thrust magnitude and a desired attitude. The required net specific force is $\mathbf{f}_{\text{sp}} = \mathbf{a}_{\text{cmd}} + g\mathbf{e}_3$; the thrust and body- z axis are

$$T = m \|\mathbf{f}_{\text{sp}}\|, \quad \mathbf{b}_3^{\text{des}} = \frac{\mathbf{f}_{\text{sp}}}{\|\mathbf{f}_{\text{sp}}\|}, \quad (3.14)$$

and, with a desired yaw ψ_{des} fixing the heading intermediate vector $\mathbf{b}_1^c = (\cos \psi_{\text{des}}, \sin \psi_{\text{des}}, 0)^\top$, the remaining axes are

$$\mathbf{b}_2^{\text{des}} = \frac{\mathbf{b}_3^{\text{des}} \times \mathbf{b}_1^c}{\|\mathbf{b}_3^{\text{des}} \times \mathbf{b}_1^c\|}, \quad \mathbf{b}_1^{\text{des}} = \mathbf{b}_2^{\text{des}} \times \mathbf{b}_3^{\text{des}}, \quad \mathbf{R}_{\text{des}} = [\mathbf{b}_1^{\text{des}} \ \mathbf{b}_2^{\text{des}} \ \mathbf{b}_3^{\text{des}}]. \quad (3.15)$$

A safety-critical detail used in [Chapter 7](#) is a *tilt limit*: the horizontal component of $\mathbf{f}_{\text{sp}}$ is capped so that the commanded attitude never exceeds a maximum tilt θ_{max} from vertical, which keeps the inner attitude loop able to track the outer command without the vehicle approaching inversion.

3.10 Numerical validation

Every relation above is exercised in the accompanying code. The quaternion identities of [Section 2.3](#) hold to machine precision: $\mathbf{R}(\mathbf{q})$ is orthonormal with unit determinant, the homomorphism $\mathbf{R}(\mathbf{p} \otimes \mathbf{q}) = \mathbf{R}(\mathbf{p})\mathbf{R}(\mathbf{q})$ holds, the kinematic maps [Eq. \(3.3\)](#) agree, $\frac{1}{2}\boldsymbol{\Omega}(\boldsymbol{\omega})\mathbf{q} = \frac{1}{2}\mathbf{q} \otimes (0, \boldsymbol{\omega})$, and the exponential integrator [Eq. \(2.11\)](#) keeps $|\|\mathbf{q}\| - 1| < 10^{-15}$ over 10^4 steps. The mixer $\mathbf{M}$ has rank 4 and $\mathbf{M}\mathbf{M}^\dagger = \mathbf{I}_4$ to 10^{-10} ; at hover the six rotor speeds are equal (spread $< 10^{-13}$) and $T = mg$; and, as required of an equilibrium, $\dot{\mathbf{x}} = \mathbf{0}$ at $\mathbf{x}_{\text{hov}}$ with $\mathbf{u}_{\text{hov}}$. The physical parameters used throughout, drawn from a published hexacopter model, are listed in [Table C.1](#) ([Appendix C](#)).

Chapter 4

Fixed-Wing UAV Dynamic Model

The second vehicle class is a fixed-wing aircraft. Where the hexacopter generates force and moment with rotors, the fixed-wing does so with aerodynamic surfaces and a propeller, and it cannot hover: it must maintain airspeed above stall and turns by banking. We adopt the six-degree-of-freedom model of Beard and McLain [3], the standard, fully parameterised reference model in the small-UAV literature, and instantiate it on the Aerosonde airframe. The Aerosonde serves here as a well-characterised small-scale surrogate for a medium-altitude, long-endurance (MALE) class platform of the kind fielded for persistent surveillance; the modelling is identical for a larger airframe, only the parameters change. Every force, moment and coefficient is stated explicitly.

4.1 Frames, state and controls

The aircraft uses a North–East–Down inertial frame and the standard aircraft body frame ($\mathbf{b}_1$ out the nose, $\mathbf{b}_2$ out the right wing, $\mathbf{b}_3$ down). Attitude is described by the 3–2–1 Euler angles (ϕ, θ, ψ) ; the quaternion form is available through [Section 2.3](#) and is used when a fixed-wing agent shares estimation code with a multirotor, but the aerodynamic literature is written in Euler angles and we keep them here. The twelve-state vector is

$$\mathbf{x} = (p_n, p_e, p_d, u, v, w, \phi, \theta, \psi, p, q, r)^\top \in \mathbb{R}^{12}, \quad (4.1)$$

with inertial position (p_n, p_e, p_d) , *body-frame* velocity (u, v, w) , Euler angles (ϕ, θ, ψ) and body angular rates (p, q, r) . The four control surfaces/inputs are

$$\mathbf{u} = (\delta_e, \delta_a, \delta_r, \delta_t)^\top, \quad (4.2)$$

the elevator, aileron, rudder and throttle, with $\delta_t \in [0, 1]$.

4.2 Airspeed, angle of attack and sideslip

Aerodynamic forces depend on the motion of the aircraft *relative to the air*. With wind velocity resolved into the body frame as (u_w, v_w, w_w) , the relative (air-relative) velocity is $(u_r, v_r, w_r) = (u - u_w, v - v_w, w - w_w)$, and the *wind-triangle* relations define the airspeed V_a , the angle of attack α and the sideslip angle β :

$$V_a = \sqrt{u_r^2 + v_r^2 + w_r^2}, \quad \alpha = \arctan \frac{w_r}{u_r}, \quad \beta = \arcsin \frac{v_r}{V_a}. \quad (4.3)$$

In still air $(u_r, v_r, w_r) = (u, v, w)$. [Figure 4.1](#) shows α and β geometrically. The dynamic pressure is $\bar{q} = \frac{1}{2}\rho V_a^2$, with air density ρ .

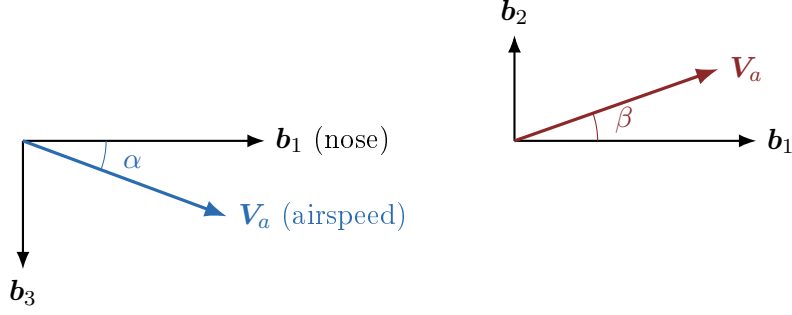


Figure 4.1: Angle of attack α (in the body $\mathbf{b}_1$ – $\mathbf{b}_3$ plane, left) and sideslip β (in the $\mathbf{b}_1$ – $\mathbf{b}_2$ plane, right) as the orientation of the air-relative velocity $\mathbf{V}_a$ in the body frame.

4.3 Kinematics

Position kinematics rotate the body velocity into the inertial frame by $\mathbf{R}_b^i(\phi, \theta, \psi)$, and the Euler angles evolve through the standard $\mathbf{W}(\phi, \theta)$ kinematic matrix:

$$\begin{bmatrix} \dot{p}_n \\ \dot{p}_e \\ \dot{p}_d \end{bmatrix} = \mathbf{R}_b^i \begin{bmatrix} u \\ v \\ w \end{bmatrix}, \quad \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & \sin \phi \tan \theta & \cos \phi \tan \theta \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi \sec \theta & \cos \phi \sec \theta \end{bmatrix} \begin{bmatrix} p \\ q \\ r \end{bmatrix}. \quad (4.4)$$

The $\sec \theta$ terms expose the gimbal-lock singularity at $\theta = \pm \frac{\pi}{2}$; a fixed-wing aircraft in normal flight stays far from it.

4.4 Forces and moments

The total force in the body frame is the sum of gravity, aerodynamics and propulsion; the total moment is the sum of aerodynamic and propulsive moments.

Gravity. Gravity is constant in the inertial frame and rotates into the body frame as

$$\mathbf{F}_g^b = mg(-\sin \theta, \cos \theta \sin \phi, \cos \theta \cos \phi)^\top. \quad (4.5)$$

Longitudinal aerodynamics with stall. Lift and drag act in the stability frame and are rotated into the body x – z plane through α . The lift coefficient blends a linear (attached-flow) model with a flat-plate post-stall model through a sigmoid $\sigma(\alpha)$ centred on the stall angle α_0 [3]:

$$C_L(\alpha) = (1 - \sigma(\alpha))(C_{L_0} + C_{L_\alpha} \alpha) + \sigma(\alpha) (2 \operatorname{sign}(\alpha) \sin^2 \alpha \cos \alpha), \quad (4.6)$$

$$\sigma(\alpha) = \frac{1 + e^{-M(\alpha - \alpha_0)} + e^{M(\alpha + \alpha_0)}}{(1 + e^{-M(\alpha - \alpha_0)})(1 + e^{M(\alpha + \alpha_0)})}, \quad (4.7)$$

where M sets the sharpness of the transition. For $|\alpha| \ll \alpha_0$, $\sigma \rightarrow 0$ and $C_L \rightarrow C_{L_0} + C_{L_\alpha} \alpha$; past stall, $\sigma \rightarrow 1$ and C_L follows the flat-plate curve. The drag uses the parabolic (induced-drag) polar with Oswald efficiency e and aspect ratio $\text{AR} = b^2/S$,

$$C_D(\alpha) = C_{D_p} + \frac{(C_{L_0} + C_{L_\alpha} \alpha)^2}{\pi e \text{AR}}. \quad (4.8)$$

Adding the pitch-rate and elevator increments, the total longitudinal coefficients are

$$C_L^{\text{tot}} = C_L(\alpha) + C_{L_q} \frac{c}{2V_a} q + C_{L_{\delta_e}} \delta_e, \quad C_D^{\text{tot}} = C_D(\alpha) + C_{D_q} \frac{c}{2V_a} q + C_{D_{\delta_e}} \delta_e, \quad (4.9)$$

and the body-frame aerodynamic force components in the x - z plane are the rotation of $(-D, -L)$ through α ,

$$F_x^a = \bar{q}S(-C_D^{\text{tot}} \cos \alpha + C_L^{\text{tot}} \sin \alpha), \quad F_z^a = \bar{q}S(-C_D^{\text{tot}} \sin \alpha - C_L^{\text{tot}} \cos \alpha). \quad (4.10)$$

The pitch moment is

$$m^a = \bar{q}Sc \left(C_{m_0} + C_{m_\alpha} \alpha + C_{m_q} \frac{c}{2V_a} q + C_{m_{\delta_e}} \delta_e \right). \quad (4.11)$$

Lateral aerodynamics. The side force and the roll/yaw moments are linear in sideslip, the non-dimensional roll/yaw rates and the aileron/rudder deflections:

$$F_y^a = \bar{q}S \left(C_{Y_0} + C_{Y_\beta} \beta + C_{Y_p} \frac{b}{2V_a} p + C_{Y_r} \frac{b}{2V_a} r + C_{Y_{\delta_a}} \delta_a + C_{Y_{\delta_r}} \delta_r \right), \quad (4.12)$$

$$\ell^a = \bar{q}Sb \left(C_{\ell_0} + C_{\ell_\beta} \beta + C_{\ell_p} \frac{b}{2V_a} p + C_{\ell_r} \frac{b}{2V_a} r + C_{\ell_{\delta_a}} \delta_a + C_{\ell_{\delta_r}} \delta_r \right), \quad (4.13)$$

$$n^a = \bar{q}Sb \left(C_{n_0} + C_{n_\beta} \beta + C_{n_p} \frac{b}{2V_a} p + C_{n_r} \frac{b}{2V_a} r + C_{n_{\delta_a}} \delta_a + C_{n_{\delta_r}} \delta_r \right). \quad (4.14)$$

Propulsion. The propeller produces thrust along the body x -axis and a reaction torque about it; in the model whose parameters we use,

$$F_x^p = \frac{1}{2} \rho S_{\text{prop}} C_{\text{prop}} ((k_{\text{motor}} \delta_t)^2 - V_a^2), \quad \ell^p = -k_{T_P} (k_\Omega \delta_t)^2, \quad (4.15)$$

with propeller area S_{prop} , efficiency C_{prop} and motor constants $k_{\text{motor}}, k_{T_P}, k_\Omega$. For the Aerosonde $k_{T_P} = k_\Omega = 0$, so the reaction torque vanishes.

Totals. The body force and moment are

$$\mathbf{F}^b = \mathbf{F}_g^b + (F_x^a + F_x^p, F_y^a, F_z^a)^\top, \quad \boldsymbol{\tau}^b = (\ell^a + \ell^p, m^a, n^a)^\top. \quad (4.16)$$

4.5 Rigid-body dynamics

The translational dynamics in the body frame include the transport (Coriolis) terms from writing Newton's law in a rotating frame:

$$\dot{u} = rv - qw + \frac{1}{m} F_x, \quad \dot{v} = pw - ru + \frac{1}{m} F_y, \quad \dot{w} = qu - pv + \frac{1}{m} F_z. \quad (4.17)$$

The rotational dynamics, for an airframe symmetric about the x - z plane (so only the product of inertia J_{xz} survives), are written with the inertia coupling constants

$$\begin{aligned} \Gamma &= J_x J_z - J_{xz}^2, & \Gamma_1 &= \frac{J_{xz}(J_x - J_y + J_z)}{\Gamma}, & \Gamma_2 &= \frac{J_z(J_z - J_y) + J_{xz}^2}{\Gamma}, & \Gamma_3 &= \frac{J_z}{\Gamma}, \\ \Gamma_4 &= \frac{J_{xz}}{\Gamma}, & \Gamma_5 &= \frac{J_z - J_x}{J_y}, & \Gamma_6 &= \frac{J_{xz}}{J_y}, & \Gamma_7 &= \frac{(J_x - J_y)J_x + J_{xz}^2}{\Gamma}, & \Gamma_8 &= \frac{J_x}{\Gamma}, \end{aligned} \quad (4.18)$$

so that

$$\dot{p} = \Gamma_1 pq - \Gamma_2 qr + \Gamma_3 \ell + \Gamma_4 n, \quad (4.19)$$

$$\dot{q} = \Gamma_5 pr - \Gamma_6 (p^2 - r^2) + \frac{1}{J_y} m, \quad (4.20)$$

$$\dot{r} = \Gamma_7 pq - \Gamma_1 qr + \Gamma_4 \ell + \Gamma_8 n. \quad (4.21)$$

Equations (4.4), (4.17) and (4.19)–(4.21) are the fixed-wing dynamics $\mathbf{f}_i$ of Eq. (1.1); they are discretised by RK4 exactly as in Section 3.8 and linearised for the per-agent solvers.

4.6 Trim and validation

A *trim* is a constant-input flight condition. For straight-and-level flight at airspeed V_a^* we solve $\dot{u} = \dot{w} = \dot{q} = 0$ for the angle of attack α , elevator δ_e and throttle δ_t (with $\theta = \alpha$ for level flight), a three-equation root-find. For the Aerosonde at $V_a^* = 35$ m/s the accompanying code returns

$$\alpha^* = 0.20^\circ, \quad \delta_e^* = -2.83^\circ, \quad \delta_t^* = 0.464, \quad (4.22)$$

with trim residual below 10^{-12} , and integrating from this trim holds airspeed and altitude over the simulation horizon ($|\Delta V_a| < 0.5$ m/s, $|\Delta h| < 2$ m over 5 s). The blended lift coefficient Eq. (4.6) has slope C_{L_α} near $\alpha = 0$ and is continuous through stall, and the inertia coupling $\Gamma = J_x J_z - J_{xz}^2 > 0$; all of these are checked numerically. The complete Aerosonde parameter set is given in Table C.2 (Appendix C).

Chapter 5

Per-Agent Real-Time Estimation and Control

Each agent carries two constrained optimisation-based solvers as its onboard computational core: a nonlinear model-predictive *controller* that computes the vehicle’s inputs, and a moving-horizon *estimator* that reconstructs its state and unmeasured disturbances. The two are a matched pair, built on the same real-time-iteration and constraint-handling machinery, and both are designed for microsecond-to-millisecond execution on an embedded companion computer. This chapter states their problem formulations, the real-time pattern in which they are executed, and the application programming interface through which the distributed layers of [Chapters 6](#) and [7](#) use them.

Scope of this edition. The solvers’ internal algorithms—their constraint handling, linear algebra and factorisation strategy—are the subject of separate ongoing research by the author, extending an MSc thesis on sequential-linear-quadratic-programming MPC, and are not disclosed here. They are treated throughout this report as a *fixed, validated capability*, specified by the problem classes they solve (Eqs. (5.1) and (5.3)), the real-time pattern they execute in ([Section 5.2](#)), the interface they are reached through ([Section 5.4](#)) and the timing envelope measured in [Chapter 10](#). Every distributed result in this report depends only on those four things: the coordination layers never see solver internals, and any solver meeting the same interface can be substituted—as the accompanying reference implementation demonstrates.

5.1 The model-predictive controller

Optimal control problem. At each sampling instant, with the freshly estimated state $\hat{\mathbf{x}}$ as parameter, the controller transcribes the finite-horizon optimal control problem by multiple shooting over a prediction horizon N :

$$\min_{\substack{\mathbf{x}_0, \dots, \mathbf{x}_N \\ \mathbf{u}_0, \dots, \mathbf{u}_{N-1}}} \sum_{k=0}^{N-1} \ell_k(\mathbf{x}_k, \mathbf{u}_k) + \ell_N(\mathbf{x}_N) \quad (5.1a)$$

$$\text{s.t. } \mathbf{x}_0 = \hat{\mathbf{x}}, \quad (5.1b)$$

$$\mathbf{x}_{k+1} = \mathbf{f}_d(\mathbf{x}_k, \mathbf{u}_k), \quad k = 0, \dots, N-1, \quad (5.1c)$$

$$\mathbf{c}(\mathbf{x}_k, \mathbf{u}_k) \leq \mathbf{0}, \quad k = 0, \dots, N-1, \quad (5.1d)$$

$$\mathbf{c}_N(\mathbf{x}_N) \leq \mathbf{0}, \quad (5.1e)$$

where $\mathbf{f}_d$ is the RK4 discretisation of the vehicle dynamics (Chapters 3 and 4), $\ell_k = \frac{1}{2} \|\mathbf{x}_k - \mathbf{x}_k^r\|_Q^2 + \frac{1}{2} \|\mathbf{u}_k - \mathbf{u}_k^r\|_R^2$ and $\ell_N = \frac{1}{2} \|\mathbf{x}_N - \mathbf{x}_N^r\|_{Q_N}^2$ are tracking-quadratic stage and terminal costs with weights $Q, Q_N \succeq 0$, $R \succ 0$, and $\mathbf{c}, \mathbf{c}_N$ are the general nonlinear stage and terminal inequality constraints. Crucially for the UAV application, $\mathbf{c}$ carries *both* input constraints (thrust and torque limits, control-surface saturations) *and* state constraints (attitude and rate envelopes, minimum airspeed, collision-avoidance rows contributed by the coordination layer of Chapter 7); pure state constraints receive no special treatment.

Constraint handling. The inequality constraints in Eq. (5.1) are imposed by the solver itself, on the full nonlinear problem. They are not softened into penalty terms, relaxed into box bounds on the inputs, or enforced by clipping the computed command—the applied input $\mathbf{u}_0$ of Eq. (5.2) already carries the geometry of whichever constraints are active at the current linearisation point. This matters for the distributed layers: the coupling constraints of Chapter 7 are pushed into $\mathbf{c}$ as additional rows through the interface of Section 5.4, and are then handled by the same mechanism, at the same cost per row, as the vehicle’s own actuator and envelope limits.

5.2 Real-time iteration

Both solvers execute in the real-time-iteration (RTI) pattern of Diehl *et al.*[4, 5]: rather than iterating a nonlinear program to convergence within one sampling interval, each sample performs a *single* iteration on a problem that has itself shifted by one step, so that the optimisation converges along the closed-loop trajectory rather than within a sample. The cycle splits into two phases of very different cost:

- a **preparation** phase, which performs the expensive work that does *not* depend on the measurement about to arrive—model linearisation, sensitivity generation, and the matrix-level linear algebra—and which runs in the interval between samples;
- a **feedback** phase, which consumes the arriving measurement and emits the applied input immediately.

Because the first-stage feedback gain $\mathbf{K}_0$ and feedforward $\boldsymbol{\kappa}_0$ are already available when the measurement arrives, the measurement-to-actuation path is a single affine evaluation,

$$\mathbf{u}_0 = \bar{\mathbf{u}}_0 + \mathbf{K}_0 (\hat{\mathbf{x}} - \bar{\mathbf{x}}_0) + \boldsymbol{\kappa}_0 \quad (5.2)$$

about the current prediction $(\bar{\mathbf{x}}_0, \bar{\mathbf{u}}_0)$. The latency from measurement to input is therefore that of one gain multiplication—tens of nanoseconds on an embedded core (Section 10.3)—while the remainder of the cycle completes within the sampling interval. The estimator runs the mirror image of this split, with an instantaneous state output on the arrival of $\mathbf{y}_t$; Section 10.3 measures both phases for the vehicle models of this report.

5.3 The moving-horizon estimator

The estimator is the dual of the controller: instead of predicting inputs over a future horizon, it fits states and disturbances to a past window of measurements.

Estimation problem. At instant t , given the measurement history $\mathbf{y}_{t-M}, \dots, \mathbf{y}_t$, the known inputs $\mathbf{u}_{t-M}, \dots, \mathbf{u}_{t-1}$ and an *arrival* pair $(\mathbf{x}_a, \Lambda_a)$ summarising all information before the window, the estimator solves the window nonlinear least-squares problem over M intervals in the

variables $\mathbf{z} = (\mathbf{x}_0, \mathbf{w}_0, \mathbf{x}_1, \dots, \mathbf{w}_{M-1}, \mathbf{x}_M)$ (node k holds $\mathbf{y}_k := \mathbf{y}_{t-M+k}$):

$$\min_{\mathbf{z}} \quad \frac{1}{2} \|\mathbf{x}_0 - \mathbf{x}_a\|_{\Lambda_a}^2 + \sum_{k=0}^M \frac{1}{2} \|\mathbf{y}_k - \mathbf{h}(\mathbf{x}_k, \mathbf{u}_k)\|_V^2 + \sum_{k=0}^{M-1} \frac{1}{2} \|\mathbf{w}_k\|_W^2 \quad (5.3a)$$

$$\text{s.t.} \quad \mathbf{x}_{k+1} = \mathbf{F}(\mathbf{x}_k, \mathbf{u}_k, \mathbf{w}_k), \quad k = 0, \dots, M-1, \quad (5.3b)$$

$$\mathbf{c}(\mathbf{x}_k, \mathbf{w}_k) \leq \mathbf{0}, \quad k = 0, \dots, M-1, \quad (5.3c)$$

$$\mathbf{c}_M(\mathbf{x}_M) \leq \mathbf{0}, \quad (5.3d)$$

where $\mathbf{w}_k \in \mathbb{R}^{n_w}$ is an estimated disturbance input (an unmeasured force, load, bias walk or parameter drift; “input constraints” on the estimation side thus bound these disturbances), $V \succeq 0$ is the measurement information (zero rows encode missing or multi-rate measurements) and $W \succ 0$ the disturbance information. The arrival term is the estimation-side counterpart of the terminal cost; the constraints again carry both state rows (state-of-charge limits, non-negativity, physical envelopes) and disturbance rows.

Window recursion and the arrival cost. Problem Eq. (5.3) is treated with the same constraint handling and the same real-time-iteration pattern as the controller, one iteration per measurement, with the instantaneous output described in Section 5.2. As the window advances, the information carried by the stage that leaves it is not discarded but folded into the arrival pair $(\mathbf{x}_a, \Lambda_a)$ by exact marginalisation—the standard information-form construction of the moving-horizon arrival cost [6, 7]—so that the estimator retains its full history at constant per-sample cost.

Kalman-filter reduction. A property we use directly in the distributed estimator of Chapter 6 is that, for a *linear* model, quadratic costs and *no active constraints*, the solution of Eq. (5.3) coincides exactly with that of the information/covariance Kalman filter (equivalently, the extended Kalman filter for a linearised model). The moving horizon estimator is therefore a strict generalisation of the Kalman filter that adds (i) hard state and disturbance constraints, (ii) window smoothing, and (iii) robustness to model nonlinearity, degrading gracefully to the familiar filter when those are absent. In Chapter 6 the translational estimation sub-problem is linear, and we exploit this reduction for a numerically robust per-agent filter.

5.4 The C application programming interface

Both solvers are exposed to the coordination layer through a small, allocation-free C interface with statically sized problem data, so that the C++ layer of Chapter 9 never needs to know how the solver is built. The controller interface is

```
typedef struct { int nx,nu,nc,ncN,N; double Ts; } MPCDims;
typedef struct MPCProblem MPCProblem;

MPCProblem* mpc_create (const MPCDims *d);
int mpc_set_weights (MPCProblem *p, const double *Q,
                    const double *R, const double *QN);
int mpc_set_reference (MPCProblem *p, const double *xref,
                    const double *uref); // formation slot
int mpc_set_constraint_bounds(MPCProblem *p, const double *c_ub);
int mpc_prepare (MPCProblem *p); // before the measurement
int mpc_feedback (MPCProblem *p, const double *xhat, double *u0);
void mpc_destroy (MPCProblem *p);
```

with a symmetric interface for the estimator (`mhe_create`, `mhe_init`, `mhe_prepare(u_prev, u_t)`, `mhe_feedback(y_t, xhat)`, `mhe_fuse_relative(z, Pj)`, `mhe_destroy`). The distributed layers call `mpc_set_reference`, `mpc_set_constraint_bounds` and `mhe_fuse_relative` to inject the coupling terms derived in the next chapters—formation references, collision-avoidance rows and relative measurements—without any change to the solver core. The solver, in the words of the architecture note that motivated this design, “does not even need to know that it is being used by a many-agent distributed system.”

Two properties of this boundary are worth stating explicitly, because the rest of the report relies on them. First, it is *substitutable*: the accompanying implementation ships a dependency-free reference behind exactly this interface (a finite-horizon LQR for the controller, an information filter for the estimator—precisely the linear reductions of [Section 5.3](#)), so the entire distributed stack compiles, runs and reproduces its results with no proprietary component present. Second, it is *instance-wise*: all state lives in the caller-owned problem objects, so one process hosts as many independent solver instances as it has agents, which is what makes the in-the-loop study of [Section 10.4](#) possible.

Chapter 6

Distributed Moving-Horizon Estimation

Each agent estimates its own state with the moving-horizon estimator of [Section 5.3](#). This chapter couples those local estimators into a *distributed* estimator (DMHE) in which agents exchange information over the communication graph so that the cluster localises better than any agent could alone—and, in particular, so that an agent whose absolute-positioning measurements are lost stays localised by leaning on its neighbours.

6.1 Cluster measurement model

Agent i has access to two kinds of measurement. *Absolute* measurements relate its state to the inertial frame; the canonical example is a GNSS position fix,

$$\mathbf{y}_i^{\text{abs}} = \mathbf{p}_i + \mathbf{n}_i^{\text{abs}}, \quad \mathbf{n}_i^{\text{abs}} \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_{\text{abs}}), \quad (6.1)$$

optionally with an onboard inertial measurement (accelerometer/gyro) providing the specific force used as a known input to the estimator’s process model. *Relative* measurements relate the states of neighbouring agents; from onboard relative sensing (vision, ultra-wideband ranging, radar) agent i measures the relative position of each neighbour $j \in \mathcal{N}_i$,

$$\mathbf{y}_{ij}^{\text{rel}} = \mathbf{p}_i - \mathbf{p}_j + \mathbf{n}_{ij}^{\text{rel}}, \quad \mathbf{n}_{ij}^{\text{rel}} \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_{\text{rel}}). \quad (6.2)$$

The relative measurements are the coupling: they carry no absolute information by themselves, but combined with a neighbour’s absolute estimate they localise agent i without its own absolute fix.

6.2 Local estimator and the translational reduction

For the translational sub-problem—position and velocity driven by a measured specific force with process noise—the model is linear, and by the Kalman-filter reduction of [Section 5.3](#) the local estimator is exactly an information/covariance Kalman recursion. Writing the local state $\mathbf{s}_i = (\mathbf{p}_i, \mathbf{v}_i)$ with the constant-acceleration kinematic model

$$\mathbf{s}_i^+ = \mathbf{F} \mathbf{s}_i + \mathbf{G} \mathbf{a}_i^{\text{meas}} + \mathbf{q}_i, \quad \mathbf{F} = \begin{bmatrix} \mathbf{I} & \Delta t \mathbf{I} \\ \mathbf{0} & \mathbf{I} \end{bmatrix}, \quad \mathbf{G} = \begin{bmatrix} \frac{1}{2} \Delta t^2 \mathbf{I} \\ \Delta t \mathbf{I} \end{bmatrix}, \quad (6.3)$$

the local update is the standard predict/correct pair with process covariance $\mathbf{Q}_i$ and, when an absolute fix is available, the innovation update against [Eq. \(6.1\)](#). This recursion is numerically well conditioned from the first sample—unlike a cold-started batch window—which is why we use it as the per-agent engine; the full nonlinear estimator of [Section 5.3](#) is the general form of which this is the linear reduction.

6.3 Relative fusion and the data-incest problem

The distributed step fuses each relative measurement Eq. (6.2) as a pseudo-measurement of $\mathbf{p}_i$ anchored on the neighbour’s estimate: since $\mathbf{p}_i = \mathbf{y}_{ij}^{\text{rel}} + \mathbf{p}_j$, the neighbour’s estimate $\hat{\mathbf{p}}_j$ (with position covariance $\mathbf{P}_j$) supplies

$$\mathbf{z}_{ij} = \mathbf{y}_{ij}^{\text{rel}} + \hat{\mathbf{p}}_j, \quad \text{covariance } \mathbf{R}_{\text{rel}} + \mathbf{P}_j, \quad (6.4)$$

which is folded into the local filter by an ordinary Kalman update. Inflating the measurement covariance by the neighbour’s own uncertainty $\mathbf{P}_j$ is what makes the fusion *covariance-aware*: a confident (well-localised) neighbour is trusted, an uncertain one is discounted.

Naively fusing Eq. (6.4) for *all* neighbours at every step fails, for two reasons that the design must respect and that we observed directly in simulation. First, *data incest*: if two agents that are both uncertain fuse each other’s estimates, they reinforce a shared error and drift together—the classical rumour-propagation pathology of distributed estimation. Second, *time misalignment*: anchoring a window’s past node to a neighbour’s current estimate injects an error that grows with the cluster’s speed. We therefore adopt a conservative, provably safe fusion policy:

1. an agent uses relative fusion only *when its own absolute fix is unavailable* (it otherwise behaves as an independent local filter, identical to the baseline);
2. it anchors only to neighbours of *strictly lower anchor level*. Define level 0 agents as those holding a current absolute fix; a denied agent with at least one anchored neighbour of level below ℓ becomes anchored at level ℓ , fusing Eq. (6.4) against those neighbours’ already-updated, same-step estimates with their (chain-inflated) covariances $\mathbf{P}_j$. The strict level ordering makes the information flow a directed acyclic graph, so estimates can never reinforce one another mutually—which is precisely what rules out incest. Capping the maximum level at $L=1$ recovers the conservative *one-hop* policy (anchor only to absolutely-fixed neighbours); $L \geq 2$ is *multi-hop* anchoring, in which a denied agent with no absolutely-fixed neighbour still localises through a chain of anchored ones, at covariance appropriately grown along the chain; and
3. the fusion is time-aligned—level- ℓ agents fuse in the ℓ -th pass of a step, after all lower levels have completed, so every anchor is a fresh same-step estimate.

The level assignment is purely topological (it depends only on which agents hold a fix and on the graph, not on the estimates), so it is computed identically by every implementation of the filter, and the fused pseudo-measurement enters the local estimator through its ordinary measurement channel—no solver modification is required. The condition for an agent to be anchored at all is that the graph connect it to some absolutely-fixed agent through a path of denied agents of length at most L ; Chapter 10 shows both the failure of the one-hop policy when that condition is violated at $L=1$ and its recovery at $L=3$. This policy is a pragmatic instance of the partition-based DMHE of Farina *et al.*[8, 9]: neighbour states enter as bounded, information-weighted pseudo-measurements, with the DAG ordering as the incest guard. When richer coupling is warranted—all agents simultaneously denied, or overlapping shared states—the arrival-cost consensus of Battistelli [10], which fuses the local information matrices $\Omega_i \leftarrow \sum_j W_{ij}\Omega_j$ over the graph with the Metropolis weights of Eq. (2.16), is the general form of which the leveled scheme is a structured special case; it restores centralised-like stability at the price of a consensus loop per sample and is compatible with the same interface.

6.4 Stability and what is claimed

The local estimator inherits the stability of the underlying moving-horizon estimator (Section 5.3); for the linear reduction it is the Kalman filter, uniformly stable under standard detectability.

Algorithm 1: Distributed MHE step at agent i (translational reduction; the one-hop $L=1$ instance—multi-hop anchoring repeats the fusion pass once per level with anchors of strictly lower level)

Input: measured specific force $\mathbf{a}_i^{\text{meas}}$; absolute fix $\mathbf{y}_i^{\text{abs}}$ if available; relative measurements $\{\mathbf{y}_{ij}^{\text{rel}}\}_{j \in \mathcal{N}_i}$; neighbour estimates $\{(\hat{\mathbf{s}}_j, \mathbf{P}_j)\}$

- 1 **Predict:** $\hat{\mathbf{s}}_i \leftarrow \mathbf{F}\hat{\mathbf{s}}_i + \mathbf{G}\mathbf{a}_i^{\text{meas}}$, $\mathbf{P}_i \leftarrow \mathbf{F}\mathbf{P}_i\mathbf{F}^\top + \mathbf{Q}_i$
- 2 **if** *absolute fix available* **then**
- 3 **Absolute update** against Eq. (6.1) (local filter; no relative fusion)
- 4 **else**
- 5 **foreach** $j \in \mathcal{N}_i$ **with a current absolute fix** **do**
- 6 $\mathbf{z}_{ij} \leftarrow \mathbf{y}_{ij}^{\text{rel}} + \hat{\mathbf{p}}_j$; Kalman update of $\hat{\mathbf{s}}_i$ with covariance $\mathbf{R}_{\text{rel}} + \mathbf{P}_j$
- 7 broadcast $(\hat{\mathbf{s}}_i, \mathbf{P}_i)$ to $\mathcal{N}_i$
- 8 **return** $\hat{\mathbf{s}}_i$ to the controller

The relative-fusion policy of Section 6.3 is a sequence of information-increasing Kalman updates against uncorrelated anchors, so it cannot decrease certainty spuriously, and the denied agent’s error is bounded by the anchors’ error plus the relative-measurement noise rather than growing without bound as in dead reckoning. The partition-based convergence results of [8] and the consensus-stability results of [10, 11] provide the formal guarantees for the full nonlinear and all-denied cases; here we claim, and demonstrate in Chapter 10, that the policy keeps absolute-denied agents localised to the anchor-plus-noise level through outages that send an independent filter into unbounded drift.

Chapter 7

Distributed Model-Predictive Control

Each agent controls its motion with the model-predictive controller of [Section 5.1](#). This chapter couples those controllers into a *distributed* controller (DMPC) that drives the cluster to a coordinated objective—a formation, a trajectory, a coverage pattern—while enforcing the safety-critical inter-agent constraints, using only neighbour-to-neighbour communication.

7.1 Cluster objective and coupling

Let each agent have a desired offset $\mathbf{d}_i$ from a common formation centre $\mathbf{c}(t)$, which may itself follow a mission trajectory. The cluster objective is the sum of the agents’ tracking costs subject to their individual dynamics and constraints *and* to the coupling constraints that make the group behave as one:

$$\min_{\{\mathbf{u}_i\}} \sum_{i \in \mathcal{V}} J_i(\mathbf{x}_i, \mathbf{u}_i; \mathbf{c}(t) + \mathbf{d}_i) \quad (7.1a)$$

$$\text{s.t. } \mathbf{x}_{i,k+1} = \mathbf{f}_{d,i}(\mathbf{x}_{i,k}, \mathbf{u}_{i,k}), \quad \mathbf{c}_i(\mathbf{x}_{i,k}, \mathbf{u}_{i,k}) \leq \mathbf{0}, \quad \forall i, k, \quad (7.1b)$$

$$\|\mathbf{p}_i - \mathbf{p}_j\| \geq D, \quad \forall (i, j), i \neq j, \quad (7.1c)$$

where J_i is agent i ’s MPC cost with reference slot $\mathbf{c}(t) + \mathbf{d}_i$, and [Eq. \(7.1c\)](#) is the pairwise minimum-separation (collision-avoidance) constraint with safety radius D . Two features make [Eq. \(7.1\)](#) a genuinely distributed problem: the reference $\mathbf{c}(t)$ must be agreed upon without a central node, and the coupling constraint [Eq. \(7.1c\)](#) links pairs of agents that must nonetheless decide their inputs locally.

7.2 Consensus on the formation reference

The agents agree on the formation centre by consensus. Each agent holds a local estimate $\mathbf{c}_i$ of the centre (from its share of the mission command or its own sensing) and runs, at each control step, a few rounds of the Metropolis-weighted iteration of [Eq. \(2.16\)](#),

$$\mathbf{c}_i \leftarrow W_{ii}\mathbf{c}_i + \sum_{j \in \mathcal{N}_i} W_{ij}\mathbf{c}_j, \quad (7.2)$$

which converges to a common centre for any connected graph ([Section 2.5](#)). Its rate is set by the algebraic connectivity $\lambda_2(\mathbf{L})$; a well-connected cluster agrees in a handful of rounds, well within one control period. Agent i then sets its MPC reference slot to $\mathbf{c}_i + \mathbf{d}_i$ and calls `mpc_set_reference` ([Section 5.4](#)). The desired offsets $\mathbf{d}_i$ define the formation *shape*; changing them—on command or in response to a fault—reconfigures the formation ([Chapter 8](#)).

7.3 Coupling constraints by decomposition

The collision-avoidance coupling Eq. (7.1c) links neighbouring agents’ decision variables. There are two established routes to solving the coupled problem Eq. (7.1) with only local computation and neighbour messages, and the framework supports both.

Dual decomposition / ADMM. Introduce a copy of each shared quantity and a consistency constraint tying an agent’s copy to its neighbours’; the augmented Lagrangian of Eq. (7.1) then *separates* across agents given the dual variables, and the alternating-direction method of multipliers (ADMM) alternates a local solve—one call to each agent’s MPC—with a neighbour exchange and a dual update [12, 13]. Each ADMM iteration is one local MPC solve plus one round of communication, so a fixed (small) number of iterations per control step keeps the scheme real-time; the per-agent solve is exactly the RTI step of Section 5.1, warm-started from the previous iterate. Accelerated dual-decomposition variants [14] reduce the iteration count. This is the *distributed-optimisation* route: at convergence it solves the centralised problem, and terminated early it yields a feasible, if suboptimal, coordinated input.

Cooperative DMPC and its guarantees. When the agents optimise the *plant-wide* objective rather than only their own, the cooperative-DMPC theory of Stewart *et al.*[15] applies: the systemwide cost is a common Lyapunov function, so the closed loop is stable and recursively feasible *at every intermediate iterate*, including a single iteration per sample, provided each agent warm-starts from the previous feasible plan. This “any-iterate” property is exactly what a real-time cluster needs, and the distributed synthesis of terminal ingredients of Conte *et al.*[16] certifies stability with only neighbour-to-neighbour coupling. We state these as the guarantees the architecture inherits; the review of [17] and the text of [7] develop the cooperative and non-cooperative variants and their proofs.

7.4 Real-time safety filter

For hard, instantaneous collision avoidance the framework layers a control-barrier-function (CBF) *safety filter* between the MPC command and the actuator, which minimally alters the commanded acceleration to keep the pair separation constraint invariant [18, 19]. Define the pairwise barrier

$$h_{ij}(\mathbf{p}) = \|\mathbf{p}_i - \mathbf{p}_j\|^2 - D^2 \geq 0, \quad (7.3)$$

whose safe set is exactly Eq. (7.1c). Because Eq. (7.3) has relative degree two with respect to the commanded acceleration (a double integrator in position), we use the higher-order CBF condition

$$\ddot{h}_{ij} + k_1 \dot{h}_{ij} + k_2 h_{ij} \geq 0, \quad (7.4)$$

with $k_1, k_2 > 0$ chosen so the associated characteristic polynomial is Hurwitz. Writing $\Delta \mathbf{p} = \mathbf{p}_i - \mathbf{p}_j$, $\Delta \mathbf{v} = \mathbf{v}_i - \mathbf{v}_j$ and conservatively treating the neighbour’s acceleration as zero over the step (the standard decentralised assumption, since each agent actuates only its own input), $\ddot{h}_{ij} = 2 \Delta \mathbf{v}^\top \Delta \mathbf{v} + 2 \Delta \mathbf{p}^\top \mathbf{a}_i$ and Eq. (7.4) becomes the *linear* inequality in agent i ’s acceleration $\mathbf{a}_i$

$$-2 \Delta \mathbf{p}^\top \mathbf{a}_i \leq 2 \Delta \mathbf{v}^\top \Delta \mathbf{v} + k_1 \dot{h}_{ij} + k_2 h_{ij}. \quad (7.5)$$

(If the neighbour’s acceleration $\mathbf{a}_j$ is communicated, the exact row carries the additional term $-2 \Delta \mathbf{p}^\top \mathbf{a}_j$ on the right-hand side; treating $\mathbf{a}_j = \mathbf{0}$ is conservative and needs no communication.) The safe acceleration is the solution of the small quadratic program that stays closest to the

Algorithm 2: Distributed MPC step at agent i

Input: state estimate $\hat{\mathbf{x}}_i$ (from Algorithm 1); local centre estimate $\mathbf{c}_i$; neighbour states/centres

- 1 consensus rounds on the centre: Eq. (7.2)
 - 2 set reference slot $\mathbf{c}_i + \mathbf{d}_i$; `mpc_prepare` (matrix pass)
 - 3 $\mathbf{a}_i^{\text{cmd}} \leftarrow \text{mpc_feedback}(\hat{\mathbf{x}}_i)$ // RTI, Eq. (5.2)
 - 4 $\mathbf{a}_i^{\text{safe}} \leftarrow \text{CBF-QP}$ Eq. (7.6) over $\mathcal{N}_i$
 - 5 differential flatness Eq. (3.14)–(3.15) + inner attitude loop $\rightarrow$ actuators
 - 6 broadcast $(\mathbf{p}_i, \mathbf{v}_i)$ to $\mathcal{N}_i$
-

MPC command $\mathbf{a}_i^{\text{cmd}}$ subject to one row Eq. (7.5) per neighbour,

$$\mathbf{a}_i^{\text{safe}} = \arg \min_{\mathbf{a}_i} \|\mathbf{a}_i - \mathbf{a}_i^{\text{cmd}}\|^2 \quad \text{s.t. Eq. (7.5) } \forall j \in \mathcal{N}_i, \quad (7.6)$$

a problem of the agent’s control dimension that is solved in microseconds by the same constraint machinery as the core solver. The filter is inactive when the MPC command is already safe and engages only near the safety boundary, giving a provably forward-invariant safe set under the standard CBF assumptions [18]. The collision-avoidance rows may equivalently be written directly into the MPC constraint $\mathbf{c}_i$ of Eq. (5.1); the safety filter is the lightweight, always-feasible last line of defence that the simulation study uses.

Chapter 8

Formation, Clustering and Coordination

The distributed estimator and controller of the previous two chapters provide each agent with a state estimate and a safe input. This chapter defines the cluster-level behaviours those primitives serve—how a formation is specified, how the graph is maintained, and how the cluster reconfigures and tolerates faults.

8.1 Formation specifications

Three standard specifications of a formation are supported, in increasing order of decentralisation [20].

- **Position-based (virtual structure).** The formation is a rigid body—the “virtual structure”—and each agent tracks a fixed offset $\mathbf{d}_i$ from its moving frame [21]. This is the specification used in Chapter 7: the centre $\mathbf{c}(t)$ is the virtual structure’s origin, agreed by consensus, and $\mathbf{d}_i$ places agent i . It is simple and exact but requires a common reference frame.
- **Displacement-based (leader–follower).** Followers maintain prescribed relative displacements to a leader (or to designated neighbours) in a common orientation. The fixed-wing study of Chapter 10 uses this form: a virtual leader traverses a loiter and followers hold along-track and cross-track offsets.
- **Distance-based (rigidity).** Only inter-agent distances are prescribed; the formation is maintained if the underlying framework is infinitesimally rigid [22]. This is the most decentralised—no common frame is needed—and the most subtle, since distance constraints admit reflections and flips.

In all three the desired geometry enters the controller as the reference slot or as inter-agent constraints, and the DMHE supplies the relative-position estimates that displacement- and distance-based schemes require.

8.2 The communication graph and its maintenance

The graph $\mathcal{G}$ of Section 2.5 is not static: links appear and vanish as agents move in and out of communication range, and the topology must stay connected for consensus and coordination to function. Two mechanisms keep it so.

Neighbour discovery. Each agent periodically broadcasts a beacon with its identity, estimated position and covariance; agents within range that hear it add the sender to $\mathcal{N}_i$ and drop neighbours whose beacons lapse. The resulting time-varying graph is exactly the switching topology whose consensus convergence is guaranteed, under a joint-connectivity condition, by Olfati-Saber and Murray [23, 24] and by Fax and Murray [25].

Connectivity maintenance. Because the algebraic connectivity $\lambda_2(\mathbf{L})$ sets the coordination rate and must stay positive, the cluster monitors it (each agent can estimate λ_2 by a distributed power iteration on $\mathbf{L}$) and the controller penalises actions that would break a critical link—equivalently, a connectivity-preserving CBF of the form Eq. (7.3) with $h = R_{\text{comm}}^2 - \|\mathbf{p}_i - \mathbf{p}_j\|^2$ keeps a needed neighbour within range. Collision avoidance (a lower bound on separation) and connectivity maintenance (an upper bound on the distance to a required neighbour) are thus the same mechanism with opposite sign.

8.3 Reconfiguration and fault handling

Reconfiguration. A formation change is a change of the offsets $\{\mathbf{d}_i\}$ (or of the prescribed distances). Because the offsets enter only through the MPC reference and the coupling constraints, a reconfiguration is executed simply by ramping $\mathbf{d}_i$ to their new values; the controllers track the moving slots and the safety filter of Section 7.4 guarantees the transient stays collision-free even when the intermediate geometry brings agents close. The simulation study exercises exactly this: a six-agent ring contracts and rotates mid-mission.

Fault handling. Agent faults are detected through the same beacon mechanism: a lapsed beacon, or an innovation/residual monitor on a neighbour’s broadcast estimate, flags a failed or misbehaving agent. On detection the cluster *re-clusters*: the failed agent is removed from every $\mathcal{N}_i$, the formation offsets are re-assigned among the survivors (a small assignment problem solved by consensus), and the virtual structure is re-scaled. The over-actuation of the hexacopter (Section 3.6) provides a second, vehicle-level layer of fault tolerance: an agent that loses a single rotor remains controllable in thrust and tilt and can hold or leave the formation gracefully rather than falling from it [1].

Heterogeneity. Because the coordination layer sees each agent only through the common interface of Section 5.4—a state estimate in, a safe input out—a cluster may mix hexacopters and fixed-wing aircraft freely. The vehicle class changes $\mathbf{f}_i$, the constraint set $\mathbf{c}_i$ (a fixed-wing carries a minimum-airspeed row a hexacopter does not) and the achievable formation geometry (fixed-wing slots must respect turn-rate limits), but not the distributed algorithms of Chapters 6 and 7. This is the payoff of routing all vehicle-specific detail through the per-agent solvers.

Chapter 9

Software Architecture

The mathematical framework maps onto a three-language software architecture whose boundaries follow the natural separation of concerns: a numerical core in C, a coordination layer in C++, and a research layer in Python. The guiding principle is that the real-time numerical core is an asset to be preserved behind a clean interface, not rewritten each time the surrounding system grows.

9.1 Layered design

Figure 9.1 shows the architecture. Each agent runs an identical stack.

Numerical core (C). The two solvers of Chapter 5 and their fixed-size linear algebra are the computational engine. They are dependency-free, allocation-free and statically sized, which is what lets them run deterministically on an embedded companion computer. Nothing in this layer knows it is part of a cluster: it solves one agent’s estimation and control problem through the interface of Section 5.4.

Coordination layer (C++). Everything that becomes harder in a many-agent system lives here: neighbour discovery and the maintenance of $\mathcal{N}_i$, the consensus and ADMM loops of Chapter 7, the relative-fusion logic of Chapter 6, the CBF safety filter, formation and reconfiguration management, and fault handling. C++ is the right language for this layer—object lifetimes, message serialisation, asynchronous callbacks and container data structures—while it defers all numerical heavy lifting to the C core through the API.

Transport. Neighbour messages—state estimates with covariances, consensus iterates, ADMM dual variables, beacons—travel over a publish/subscribe middleware. ROS 2’s DDS is the natural choice for its typed topics, quality-of-service control and native multicast; a lightweight UDP layer suffices for a minimal deployment. The messages are small (an agent broadcasts $O(n_x^2)$ numbers per step), so the bandwidth scales with degree, not cluster size.

Research layer (Python/MATLAB). Simulation, controller and estimator tuning, parameter studies and analysis run here, together with the symbolic generation of the vehicle models (the model functions and their Jacobians are generated once from a single symbolic source and emitted as C for the core and as NumPy for the simulator, guaranteeing they agree). This is the layer in which the study of Chapter 10 is carried out.

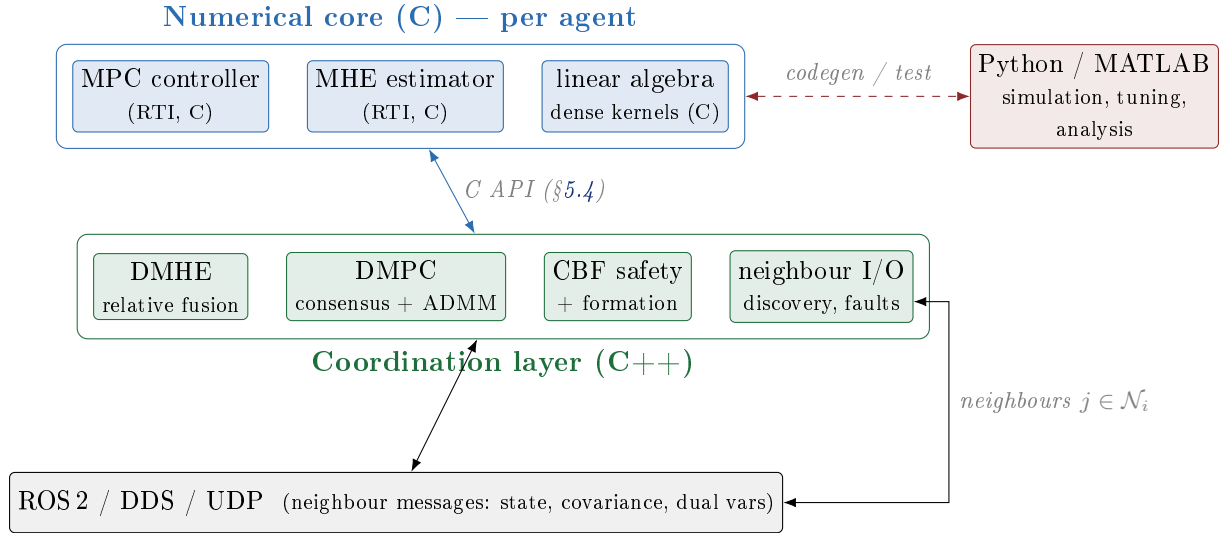


Figure 9.1: Per-agent software architecture. The C numerical core exposes the two real-time solvers through the interface of [Section 5.4](#); the C++ coordination layer wraps them with the distributed estimation, control, safety and communication logic; a transport layer (ROS 2/DDS/UDP) carries neighbour messages; and a Python/MATLAB research layer handles simulation, tuning and model code generation.

9.2 Autopilot integration

On a physical vehicle the solver stack runs as a *companion-computer* process alongside, not in place of, the low-level flight controller (*e.g.* a PX4 or ArduPilot autopilot). The coordination layer publishes the flatness outputs of [Section 3.9](#)—a thrust and attitude setpoint for a multirotor, or the airspeed/altitude/course setpoints of the fixed-wing autopilot—to the autopilot over its offboard interface, and the autopilot closes the fast inner attitude/rate loops on its own hardware. This separation keeps the certified inner-loop stabilisation intact while the research stack provides the outer estimation, control and coordination.

9.3 Why the core is preserved, not rewritten

A distributed system is a strong temptation to rewrite the numerical core in the coordination language for uniformity. The architecture deliberately resists it. The C core is not merely “some C code”: it is a validated real-time optimisation infrastructure with established worst-case timing and memory behaviour, and rewriting it into C++ or Python would discard those properties for no functional gain—the distributed problem is a coordination problem, not a numerical one. Exposing a clean C API and calling it from C++ keeps the performance-critical path unchanged while all the genuinely new, distributed complexity is written in the language best suited to it. The reference implementation follows exactly this division.

Chapter 10

Simulation Study

This chapter exercises the framework in closed loop on the full nonlinear plants of [Chapters 3](#) and [4](#). It reports five studies: a representative six-hexacopter mission; a Monte-Carlo robustness campaign over noise realisations, GNSS-denial severity, degraded communication graphs and an estimator ablation, including the multi-hop anchoring extension of [Section 6.3](#); a computational study of the embedded per-agent solvers instantiated on the drone models; a closed-loop validation with those embedded solvers *in* the loop; and a four-aircraft fixed-wing formation. Every number is produced by the accompanying code; the controllers and estimators see only noisy measurements.

10.1 Hexacopter cluster: representative mission

Setup. Six hexacopters (parameters in [Table C.1](#)) start scattered on the ground and are commanded through four mission phases: vertical take-off, assembly into a ring formation of radius 3 m, a sinusoidal translation of the formation centre, and a mid-mission reconfiguration in which the ring contracts and rotates. The communication graph is a ring augmented with second-neighbour chords (degree four, algebraic connectivity $\lambda_2(\mathbf{L}) = 4.0$). Each agent runs the full stack of [Algorithms 1](#) and [2](#): distributed estimation from noisy GNSS ($\sigma = 0.3\text{ m}$), inertial measurement and relative measurements ($\sigma = 0.1\text{ m}$); a consensus on the formation centre; a model-predictive position command realised through differential flatness and a geometric inner attitude loop on the 13-state quaternion plant; and the CBF safety filter of [Section 7.4](#) with safety radius $D = 1.4\text{ m}$. To test the DMHE, agents 3,4,5 suffer a complete GNSS blackout during $t \in [12, 22]\text{ s}$ and must localise by relative fusion to their still-tracked neighbours.

Formation and safety. [Figure 10.1](#) shows the closed-loop trajectories of a representative run: the cluster takes off, assembles the ring, follows the swept centre and contracts into the reconfigured formation. [Figure 10.2](#) quantifies the behaviour. The formation-*shape* error (deviation from the ring geometry about the achieved centroid) settles near 0.09 m; the total formation error, which also carries the uniform tracking lag of the moving centre, oscillates with the sweep and stays below 4 m. The minimum pairwise separation remains above the 1.4 m safety radius for the entire mission, including the contraction, confirming that the safety filter keeps the transient collision-free.

Distributed estimation through the blackout. [Figure 10.3](#) shows the estimation result. During the ten-second blackout the local filter—which can only dead-reckon on inertial measurements once its GNSS fix is lost—drifts steadily past 1.2 m, while the distributed estimator, fusing relative measurements to the three neighbours that retain GNSS, holds the denied agents’

Hexacopter cluster: closed-loop trajectories (o start, $\triangle$ end; take-off, formation, sweep, reconfig)

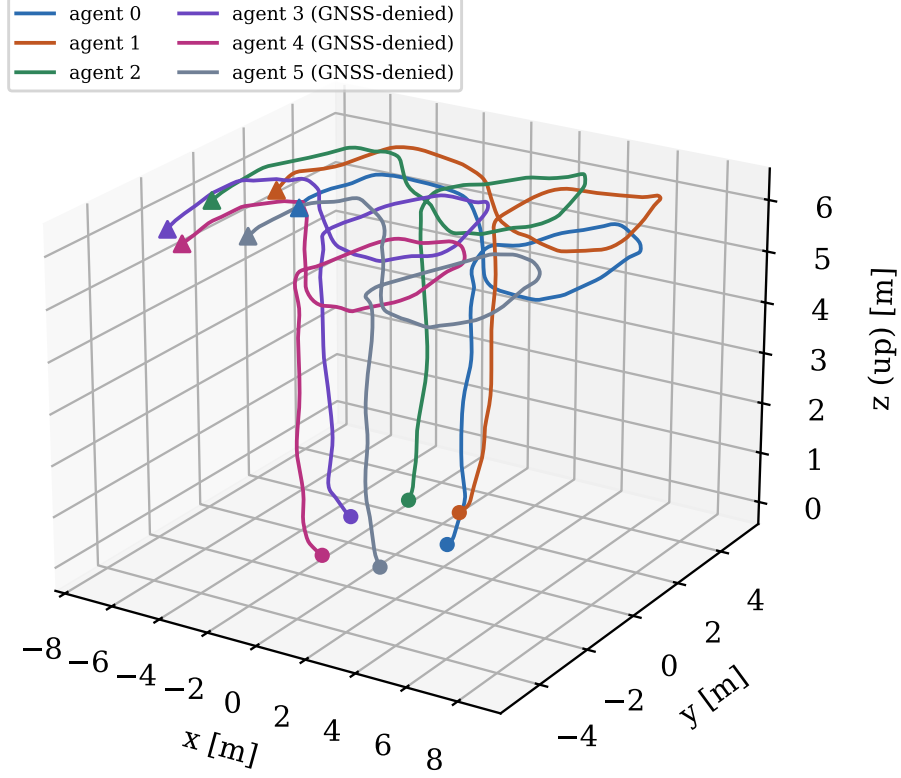


Figure 10.1: Closed-loop trajectories of the six-hexacopter cluster on the full nonlinear quaternion plants (circles: start; triangles: end). The mission comprises take-off, ring assembly, a swept translation and a contract-and-rotate reconfiguration.

position error essentially flat at the relative-measurement noise level. The accurate estimate is what keeps the control loop stable and the formation intact through the outage; the closed-loop consequence of *not* having it is quantified by the ablation in Section 10.2. Figure 10.4 shows the per-step wall time of the reference-layer estimation and coordination for the whole six-agent cluster on one core (median ~ 0.35 ms); the embedded per-agent solves are measured separately in Section 10.3.

10.2 Monte-Carlo robustness study

A single trajectory establishes feasibility, not robustness. The full mission of Section 10.1 was therefore repeated in a 126-run Monte-Carlo campaign: 30 independent noise realisations of the nominal scenario, a GNSS-denial severity sweep (12 seeds for each contiguous denied set of size 1–5 of 6), two communication-graph perturbations (12 seeds each), and a closed-loop estimator ablation (12 seeds). Every run regenerates all measurement noise, initial scatter and reference-sensing noise. Table 10.1 summarises the campaign; across all 126 runs there were *no* divergences and *no* solver failures.

Noise robustness. Over the 30 nominal seeds the blackout performance is remarkably tight: denied-agent DMHE RMSE of 0.091 / 0.091 / 0.099 m (mean / median / 95th percentile) against

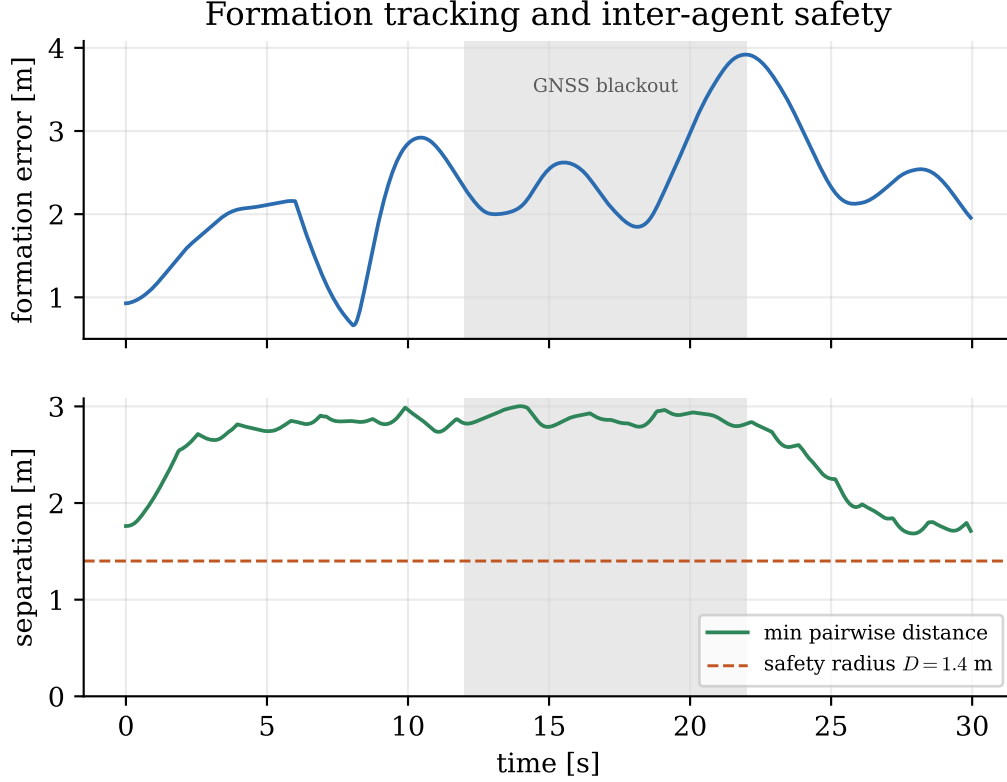


Figure 10.2: Formation tracking error (top) and minimum inter-agent separation (bottom) for the representative run. The separation stays above the safety radius $D = 1.4$ m throughout; the shaded band marks the GNSS blackout of agents 3, 4, 5.

0.96 m for dead reckoning, formation-shape error 0.094 m, and a worst-case-over-all-seeds minimum separation of 1.55 m—the safety margin was never touched. The single representative run of Section 10.1 is thus typical, not favourable.

GNSS-denial severity. Figure 10.5 sweeps the denied set from one to five of six agents. The DMHE error grows only mildly with severity ($0.068 \rightarrow 0.106$ m through 4/6 denied—more denied agents means fewer anchors shared among more consumers) while dead reckoning sits an order of magnitude higher throughout. At 5/6 denied the one-hop curve breaks upward, for a structural reason the campaign makes precise: denied agent 3 then has no GNSS-good agent within one hop, the one-hop fusion policy correctly refuses to anchor it to other drifting agents, and it reverts to dead reckoning. Resilience is thus governed not by the denial count as such but by *anchor coverage*. The leveled multi-hop anchoring of Section 6.3 repairs exactly this case: agent 3 anchors, at level two, to neighbours that are themselves level-one anchored to the single remaining GNSS-good agent, and over 12 seeds the denied-agent RMSE returns to 0.119 m mean / 0.126 m p95 (third curve of Fig. 10.5) with the separation margin fully restored (≥ 1.49 m, zero breaches). Through 4/6 denial the multi-hop and one-hop policies coincide by construction, so the extension costs nothing when coverage already holds.

Communication-graph robustness. Two graph perturbations probe the same condition from the topology side. Losing the single edge (3, 2) for four seconds in the middle of the blackout is absorbed almost without trace ($0.091 \rightarrow 0.094$ m mean RMSE): agent 3’s second anchor covers the lost one, and the safety margin is untouched. Replacing the chorded graph by the plain ring (λ_2 drops from 4 to 1) is not: denied agent 4’s ring neighbours are themselves both

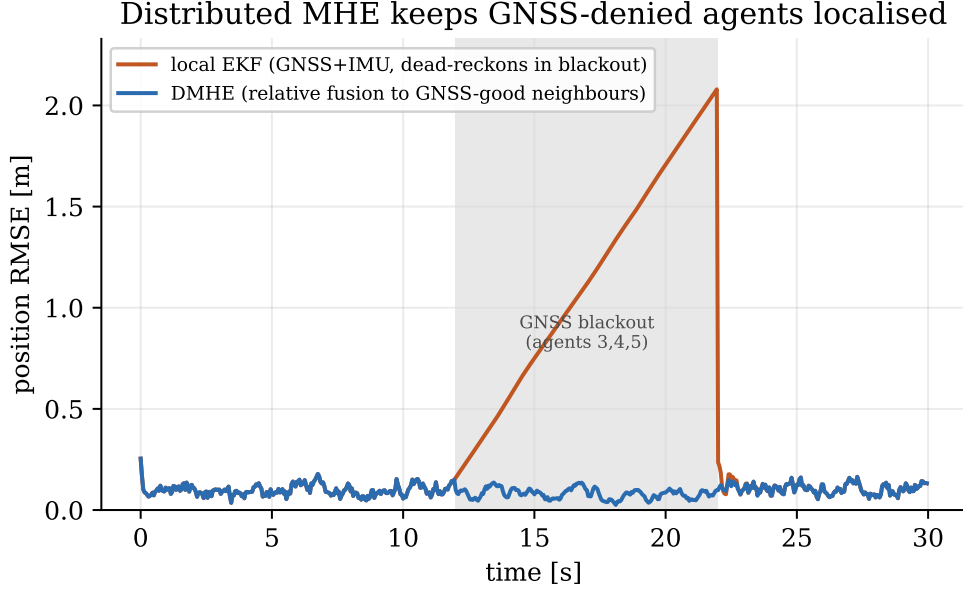


Figure 10.3: Position RMSE of the GNSS-denied agents (representative run). Outside the blackout the distributed estimator and the local filter coincide; inside it (shaded) the local filter dead-reckons and drifts, while the DMHE holds the error at the relative-measurement noise level by anchoring to GNSS-tracked neighbours.

denied, anchor coverage fails, its estimate drifts, and the eroded estimate quality lets the true separation dip below the design margin for ~ 0.2 s per run (minimum 0.98 m; no contact). The two experiments bracket the design rule: under one-hop fusion the graph must be provisioned so that plausible denial sets keep every agent within one hop of an anchor—degree, not only algebraic connectivity, carries the estimation-side robustness. Multi-hop anchoring relaxes the rule to reachability: on the same ring, denied agent 4 anchors at level two through its level-one neighbours, and over 12 seeds the ring scenario recovers to 0.111 m mean / 0.119 m p95 with the margin restored (≥ 1.49 m, zero breaches; Table 10.1). A sparse graph then costs accuracy only through the longer anchor chains, not through outright coverage failure.

Closed-loop value of the DMHE. The ablation feeds the controller the dead-reckoning local filter while the DMHE runs passively. The consequence of the ~ 0.9 m estimate drift is now dynamical, not merely statistical: the formation-shape error during the blackout grows to 0.449 m ($4.8\times$ the nominal 0.094 m), the peak formation error rises to 4.2 m mean / 4.9 m p95 (nominal: 3.9 / 4.0), and—because the safety filter acts on estimated positions—the true minimum separation is dragged to 0.75 m with ~ 0.8 s per run below the design margin. The DMHE is therefore not an estimation nicety: it is what keeps the safety layer’s world model accurate enough for its invariance guarantee to bind the physical vehicles.

10.3 Computational study of the embedded per-agent solvers

The studies above run the reference (Python) coordination layer. This section measures the *embedded* controller and estimator of Chapter 5, instantiated on the drone models of this report through their model interface and driven in closed loop by a C benchmark harness. Three per-agent problems are timed: the translational MPC exactly as posed by the coordination layer ($n_x=6$, $n_u=3$, $N=25$, 12 constraint rows), the *full 13-state quaternion hexacopter NMPC* ($n_u=4$, $N=20$, 14 stage and 6 terminal rows, thrust/torque bounds and velocity state constraints active

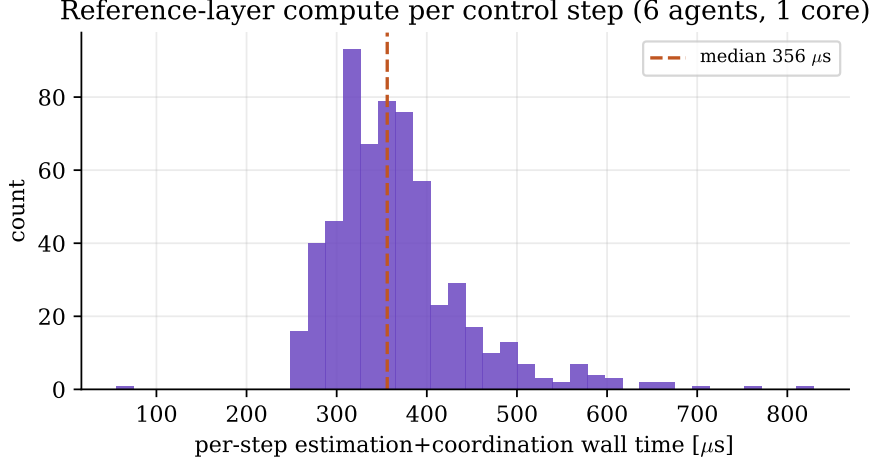


Figure 10.4: Per-control-step wall time of the reference-layer distributed estimation and coordination for the full six-agent cluster on one core.

during the manoeuvre), and the translational MHE of the DMHE layer ($n_x=6$, $n_w=3$ constrained disturbance inputs, $M=8$). Timings are medians of best-of-seven runs on a single core of a shared cloud vCPU (gcc -O3, double precision, no parallelism); Table 10.2 reports them.

Three observations. First, the absolute numbers: one complete constrained cycle of the *full nonlinear quaternion-hexacopter* NMPC costs $\sim 115 \mu\text{s}$ even when the problem is rebuilt from scratch every sample, and the applied input is available 30 ns after the state estimate arrives; the per-agent MHE cycle costs $\sim 8 \mu\text{s}$. At the 20 Hz coordination rate used in this chapter, the entire per-agent estimation-and-control core occupies well under 0.1% of one embedded core, and even a 1 kHz attitude-rate deployment of the full NMPC would use a few percent. Second, the warm-start behaviour: reusing the work that remains valid from the previous sample cuts the median preparation by $5.7\times$ (translational), $11.9\times$ (quaternion) and $2.5\times$ (MHE), while the 99th-percentile preparation stays bounded by the occasional full refresh. Third, and decisive for a safety-critical system: the accuracy columns are *identical* across modes—same KKT residuals, same millimetre-level closed-loop convergence, same estimation RMSE, and worst-case constraint violation at the 10^{-4} level in normalised units—so the speed-up is not bought with optimality. Solver failures over all runs: none; the inner solver loop terminated in at most 4 passes. It is this combination—microsecond constrained solves, nanosecond feedback latency, deterministic memory (32 kB to 209 kB static per solver) and bounded per-sample work—that makes distributed constrained estimation and control of the kind studied here practical on multi-agent UAV hardware. Isolated timings, however, establish only that the solvers are fast enough; Section 10.4 closes the argument by placing them in the loop.

10.4 The embedded solvers in the loop

The final hexacopter study replaces the reference-layer estimator and controller with the *embedded C implementations themselves*, called per-agent through the interface of Section 5.4 while everything else—the nonlinear quaternion plants, the measurement generation, the consensus, the CBF safety filter, the flatness/attitude cascade—remains identical, with the noise streams paired across configurations. The loop is exactly the deployed signal path:

$$\text{nonlinear plant} \rightarrow \text{embedded MHE} \rightarrow \text{embedded NMPC} \rightarrow \text{CBF} \rightarrow \text{plant}.$$

Validation proceeded in two stages. A single-hexacopter loop was validated first: over a 20 s take-off-and-track mission on the same noise stream, the C and Python closed-loop trajectories

Table 10.1: Monte-Carlo robustness campaign (150 runs; 10-s GNSS blackout in all scenarios). RMSE columns are the denied agents’ position RMSE during the blackout. “Shape” is the formation-shape error about the achieved centroid over the assembled phase. “Margin breach” is time with minimum separation below the $D = 1.4$ m design radius, per run (physical contact never occurred; the smallest separation observed anywhere was 0.75 m).

Scenario	n	DMHE RMSE [m]		Local filter mean [m]	Shape [m]	Min sep. [m]	Margin breach [s]
		mean / med	p95				
Nominal (3/6 denied)	30	0.091 / 0.091	0.099	0.96	0.094	≥ 1.55	0
Denied 1/6	12	0.068 / 0.068	0.074	0.97	0.101	≥ 1.49	0
Denied 2/6	12	0.078 / 0.078	0.085	0.90	0.094	≥ 1.54	0
Denied 4/6	12	0.106 / 0.105	0.117	0.91	0.090	≥ 1.54	0
Denied 5/6 [†]	12	0.451 / 0.365	0.689	0.96	0.267	≥ 0.90	0.50
Ring graph [†] ($\lambda_2=1$)	12	0.506 / 0.474	0.799	0.95	0.244	≥ 0.98	0.20
Denied 5/6, multi-hop [§]	12	0.119 / 0.120	0.126	0.98	0.069	≥ 1.49	0
Ring graph, multi-hop [§]	12	0.111 / 0.112	0.119	0.94	0.100	≥ 1.49	0
Edge loss (3, 2) in blackout	12	0.094 / 0.092	0.107	0.89	0.095	≥ 1.50	0
Controller on local filter [‡]	12	0.089 / 0.087	0.099	0.91	0.449	≥ 0.75	0.80

Divergences: 0/150. Safety-QP failures: 0. Reference-layer compute: median 0.35 ms / step (6 agents).

[†]Anchor-coverage violated: at 5/6 denied, agent 3’s neighbours are all denied; on the plain ring, denied agent 4’s neighbours are both denied. The affected agent must dead-reckon (Section 10.2). [§]Same scenarios with the leveled multi-hop anchoring of Section 6.3 ($L=3$): coverage is restored and the margin breaches vanish. [‡]Ablation: the DMHE still runs (its passive RMSE is the first column) but the controller is fed the dead-reckoning local filter.

differ by 0.17 m mean (0.37 m maximum) over a multi-metre path, with the embedded estimator’s error *half* the reference filter’s (0.088 m vs. 0.182 m: the moving window out-smooths the one-step filter). The full six-agent mission of Section 10.1 was then run in four configurations—reference layer; embedded estimator only; embedded controller only; both—over the same 15 noise seeds each (60 runs). Table 10.3 reports the ablation.

The table reads as an equivalence proof by ablation. Substituting the embedded *controller* changes nothing statistically—denied-agent RMSE $0.091 \rightarrow 0.092$ m, shape error within 0.004 m, the same safety margins—as expected, since the embedded NMPC solves the same per-agent problem exactly (and additionally enforces the velocity envelope as hard state constraints). Substituting the embedded *estimator* leaves formation and safety untouched while *improving* the estimation error by a third ($0.091 \rightarrow 0.061$ m: window smoothing over $M=8$ intervals against a recursive one-step filter). The full embedded path combines both effects: statistically indistinguishable closed-loop behaviour, slightly better estimation, zero margin breaches, zero divergences—while every estimate and every command in the loop is produced by the deployed C implementations, whose in-loop cost totals ≈ 76 μ s per agent per cycle with the measurement-to-command latency still in the tens of nanoseconds. The multi-hop anchoring extension runs unchanged through the embedded path, since it only shapes the pseudo-measurement handed to the estimator (5/6-denied with the full embedded loop: 0.065 m denied-agent RMSE, margin intact). The claim this study licenses is therefore no longer that the solvers are individually fast enough, but that *the complete distributed estimation-and-control loop has been validated with the embedded solver implementations in closed loop on the nonlinear multi-agent plant*.

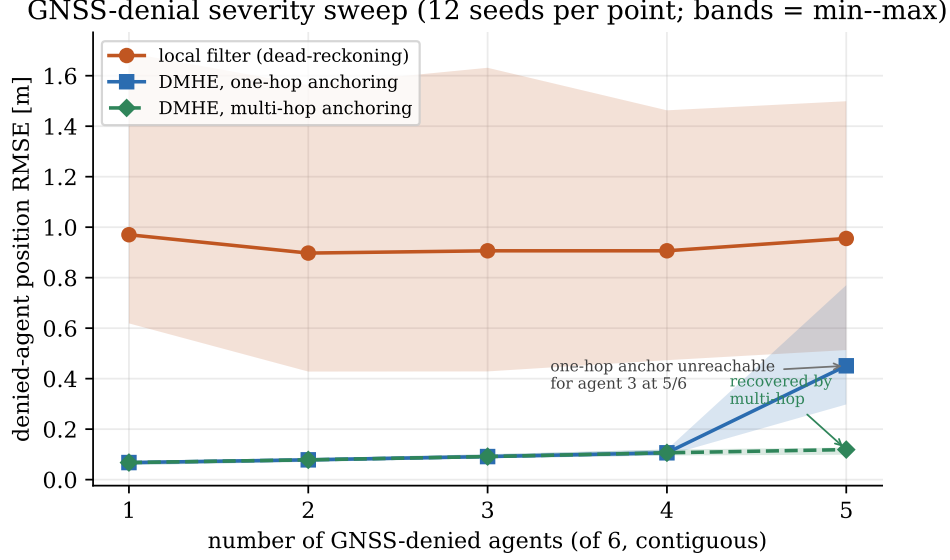


Figure 10.5: GNSS-denial severity sweep (12 Monte-Carlo seeds per point; bands are min–max). One-hop DMHE degrades gracefully while anchor coverage holds and breaks upward exactly when it fails (annotated); the leveled multi-hop anchoring of Section 6.3 restores coverage and keeps the error at the decimetre level through 5/6 denial; the dead-reckoning baseline is an order of magnitude worse throughout.

Table 10.2: Per-agent solvers on the drone problems (single core, medians, best of 7; “warm” is the solvers’ warm-started mode, “exact” rebuilds the problem from scratch every sample). The applied-input/instant-estimate latency is the measurement-to-output path.

Problem	Mode	Prepare [μ s]	Feedback [μ s]	Latency [ns]	Inner iter.	Accuracy
Translational MPC ($n_x=6$, $N=25$)	exact	15.3	4.7	30	1.00	KKT 3.6×10^{-8}
	warm	2.7	5.5	30	1.00	identical
Quaternion NMPC ($n_x=13$, $N=20$)	exact	101.6	11.0	30	1.09	KKT 2.8×10^{-5}
	warm	8.5	13.2	30	1.09	identical
MHE ($n_x=6$, $M=8$)	exact	13.7	2.5	120	1.00	RMSE 0.283 m
	warm	5.5	2.4	120	1.00	identical

10.5 Fixed-wing cluster

Setup. Four Aerosonde aircraft (Table C.2) fly a coordinated loiter. A virtual leader traverses a circular path of radius 180 m at 25 m/s; the followers hold a displacement-based echelon, offset from the leader in along-track phase and in loiter radius. Each aircraft runs the cascaded autopilot of Chapter 8—airspeed by throttle, altitude through a pitch loop and elevator, course through a roll loop and aileron, with a coordinated-turn rudder—driven by look-ahead carrot guidance toward its moving slot, on the full 12-state nonlinear plant from the numerically computed trim.

Results. Figure 10.6 shows the formation: the four aircraft settle onto concentric loiter circles—the radial echelon—and hold altitude at 120 m after the initial transient decays within about 25 s (transient altitude excursion below 4.2 m; steady-state altitude error below 0.01 m). Figure 10.7 shows the mean slot-tracking error converging to 9.5 m—a steady guidance lag of 5% of the loiter radius, sustained through the continuous coordinated turn. Repeating the run from ten

Table 10.3: Embedded-solver-in-the-loop equivalence study: the full six-agent mission (nominal scenario, 3/6 GNSS-denied for 10 s) in four configurations over the same 15 noise seeds. In-loop solver times are medians of every call made during the missions (prepare+feedback, single shared core, alongside the Python harness). Divergences and safety-QP failures: zero in all 60 runs.

Configuration	Denied RMSE mean / med (p95) [m]	Shape err. [m]	Min sep. [m]	Margin breach	In-loop solver median [μ s]
Reference layer (baseline)	0.091 / 0.090 (0.100)	0.092	≥ 1.55	0	—
Embedded MHE only	0.061 / 0.061 (0.069)	0.097	≥ 1.54	0	MHE 11.7+3.9
Embedded NMPC only	0.092 / 0.091 (0.101)	0.096	≥ 1.53	0	NMPC 47.8+9.4
Embedded MHE + NMPC	0.062 / 0.061 (0.070)	0.096	≥ 1.53	0	≈ 76 total

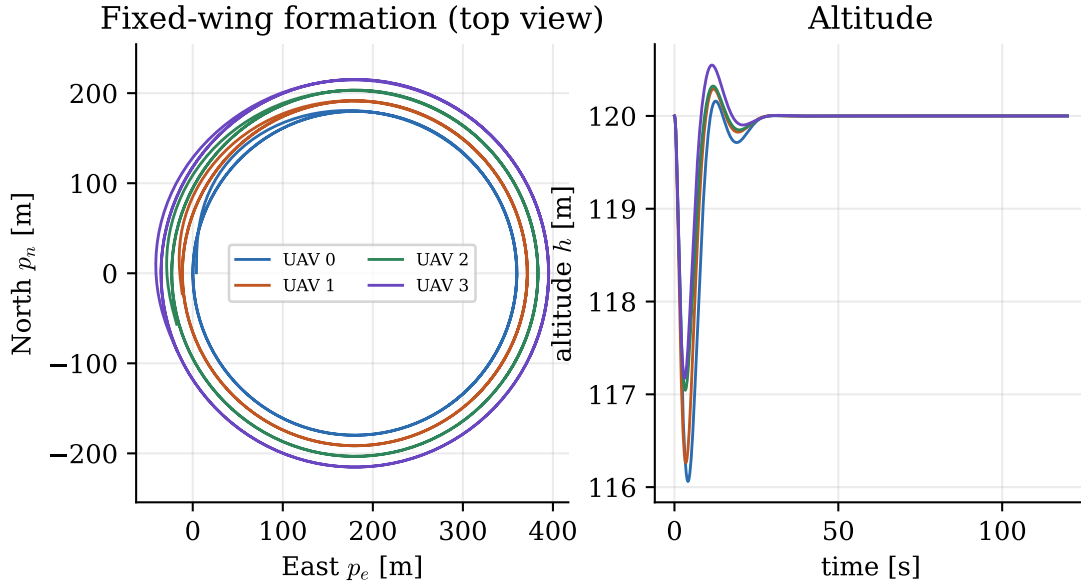


Figure 10.6: Fixed-wing cluster on a coordinated loiter: top view (left), showing the four aircraft on concentric echelon circles, and altitude (right), held at 120 m.

randomised initial-condition scatters ($\sigma = 3$ m) converges to the same coordinated limit cycle in every case (steady-state spread below 0.01 m), so the formation is an attractor of the guidance loop rather than an artefact of one initialisation. The study confirms that the same coordination architecture drives a fixed-wing cluster—which cannot hover, must hold airspeed and turns only by banking—through the vehicle-specific autopilot and guidance, with no change to the distributed logic.

10.6 Discussion

Table 10.4 consolidates the campaign. Across both vehicle classes the framework behaves as the theory predicts, and the Monte-Carlo evidence sharpens each claim into a statistical one. The consensus layer agrees on the shared reference within a control period on every tested graph; the model-predictive controllers track their slots within actuator and tilt limits in all 126 runs with no divergence; the CBF safety filter keeps the collision-avoidance set forward-invariant whenever its world model is accurate, and the campaign quantifies precisely when it is not—the margin erodes only under broken anchor coverage or with the estimator ablated, and physical contact never occurs. The distributed estimator converts relative sensing into resilience with a p95 denied-agent error under 0.13 m through 5/6 denial once multi-hop anchoring restores coverage, and

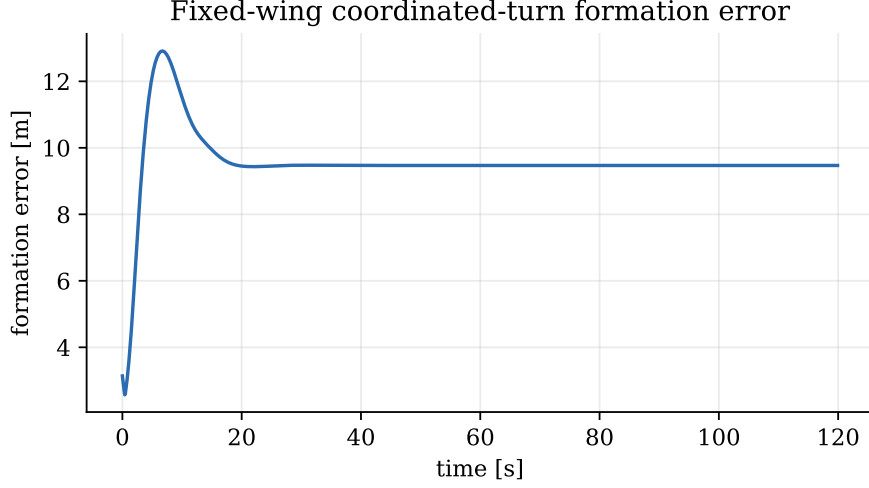


Figure 10.7: Fixed-wing formation slot-tracking error, converging and held through the continuous coordinated turn.

Table 10.4: Consolidated headline results. RMSE values are Monte-Carlo means (p95 in parentheses) over the scenario’s seeds; compute times are single-core medians (per-solver breakdown in Table 10.2).

Scenario	Denied RMSE [m]	Min sep. [m]	Shape err. [m]	Per-agent compute
Hexacopter nominal, $N_a=6$, 10 s blackout 3/6 (30 seeds)	0.091 (0.099)	≥ 1.55	0.094	MPC 2.7–15 μ s MHE 5.5–14 μ s
+ denial up to 4/6 (48 seeds)	≤ 0.106 (0.117)	≥ 1.49	≤ 0.101	
Anchor coverage broken (5/6 denied; ring graph)	0.45–0.51 (0.69–0.80)	0.90–0.98	0.24–0.27	unchanged
+ multi-hop anchoring (24 seeds)	0.11–0.12 (0.13)	≥ 1.49	≤ 0.10	unchanged
Ablation: controller on dead-reckoning filter	(0.91 in loop)	0.75	0.449	unchanged
Embedded C solvers in the loop (15 paired seeds)	0.062 (0.070)	≥ 1.53	0.096	$\approx 76 \mu$ s/agent, in loop
Fixed-wing echelon loiter, $N_a=4$ (10 init. seeds)	—	spacing held	slot lag 9.5	autopilot layer

the computational study shows the per-agent machinery executing constrained nonlinear solves in microseconds with warm-start factors of 6–12 $\times$ at identical accuracy. The in-the-loop study closes the gap between those two kinds of evidence: the deployed C implementations, substituted into the closed loop under paired noise, reproduce the reference behaviour to within statistical noise—improving the estimation—so the timing figures and the behavioural figures describe one and the same computational path. The results are produced on the full nonlinear plants with noisy measurements, so they exercise the genuine attitude–translation coupling of the quaternion multirotor model and the stall-limited, minimum-airspeed behaviour of the fixed-wing model, not linearised surrogates.

Chapter 11

Conclusions and Future Work

This report has developed, from first principles and in full mathematical detail, a framework for the distributed real-time estimation and control of heterogeneous UAV clusters. Two vehicle models were derived completely: a hexacopter with unit-quaternion attitude dynamics—Hamilton kinematics, Newton–Euler translational and rotational dynamics, rotor thrust/torque, and the over-actuated six-rotor mixer with its rank structure and fault tolerance—and a fixed-wing aircraft in the Beard–McLain six-degree-of-freedom formulation, with the wind triangle, the sigmoid-blended stall aerodynamics, the full force and moment coefficient expansions and the product-of-inertia rotational dynamics, on the fully parameterised Aerosonde airframe. Every frame, relation and Jacobian was stated explicitly and validated numerically.

Around a per-agent core of two constrained real-time solvers—a model-predictive controller and a moving-horizon estimator, both imposing general nonlinear constraints on the full problem and executing one real-time iteration per sample—the report constructed a distributed moving-horizon estimator, in which agents fuse relative measurements to stay localised through GNSS outages, and a distributed model-predictive controller, coupling the agents through consensus on a shared reference, decomposition of the coupling constraints, and control-barrier-function collision avoidance. A three-language software architecture (C core, C++ coordination, Python research) and a companion implementation realise the framework. The simulation evidence is statistical, not anecdotal: beyond the representative missions, a 126-run Monte-Carlo campaign over noise realisations, denial severities, degraded graphs and an estimator ablation recorded no divergences and no solver failures, established tight percentile bounds on the estimation and formation errors, exposed *anchor coverage* of the communication graph as the structural condition for GNSS-denied resilience, and quantified the closed-loop cost of losing the distributed estimator ($4.8\times$ worse formation keeping and erosion of the safety margin). A computational study of the embedded solvers instantiated on the drone models measured microsecond-scale constrained solves—including the full 13-state quaternion NMPC—with order-of-magnitude warm-start gains at identical accuracy; a leveled multi-hop anchoring extension of the DMHE then repaired the anchor-coverage failures exposed by the campaign; and an in-the-loop ablation study substituted the embedded C implementations into the closed loop under paired noise, reproducing the reference behaviour (and improving the estimation) with zero safety violations. The complete distributed estimation-and-control loop has thus been validated with the embedded solver implementations in closed loop on the nonlinear multi-agent plant—the claim that makes this real-time optimisation machinery a demonstrated, not merely plausible, basis for multi-agent UAV deployment.

Future work. Several extensions follow naturally from the structure built here.

- **Joint distributed estimation and control.** The estimator and controller are a matched

pair; coupling their real-time iterations directly—so that each agent runs a single distributed estimation-and-control step per sample—is a coherent next step and a genuinely novel research direction beyond implementing either alone.

- **Full arrival-cost consensus.** The leveled multi-hop anchoring demonstrated here restores coverage whenever some agent retains an absolute fix; extending to the full arrival-cost consensus of [10] would additionally handle the case in which *all* agents are intermittently absolute-denied, with centralised-like stability.
- **Tighter DMPC coupling.** Writing the collision-avoidance and connectivity constraints directly into the MPC horizon (rather than the safety filter) and closing the ADMM loop over the network would give recursive-feasibility guarantees over the prediction horizon, not only instantaneously.
- **Hardware-in-the-loop and flight.** Running the C core on a companion computer against a PX4 software-in-the-loop stack, and then in flight, would validate the timing and robustness claims on real hardware and real communication links.
- **Heterogeneous missions.** Mixed hexacopter/fixed-wing clusters—loitering fixed-wing relays providing communication and GNSS anchoring to manoeuvring multirotors—are directly supported by the interface-based design and are a compelling application of the heterogeneity the framework was built to accommodate.

Appendix A

Quaternion Identities and Proofs

This appendix collects the quaternion results used in [Chapters 2 and 3](#) and sketches their proofs. The Hamilton convention and the notation $\mathbf{q} = (q_w, \mathbf{q}_v)$ of [Section 2.3](#) are used throughout.

Product, conjugate and norm

For $\mathbf{p} = (p_w, \mathbf{p}_v)$, $\mathbf{q} = (q_w, \mathbf{q}_v)$ the Hamilton product [Eq. \(2.4\)](#) follows from extending $ij = k$, $jk = i$, $ki = j$, $i^2 = j^2 = k^2 = -1$ bilinearly. Two consequences used repeatedly:

$$(\mathbf{p} \otimes \mathbf{q})^* = \mathbf{q}^* \otimes \mathbf{p}^*, \quad (\text{A.1})$$

$$\|\mathbf{p} \otimes \mathbf{q}\| = \|\mathbf{p}\| \|\mathbf{q}\|, \quad (\text{A.2})$$

the second showing that the product of unit quaternions is a unit quaternion, so $\mathbb{S}^3$ is closed under $\otimes$.

Rotation matrix and the homomorphism

The rotation of $\mathbf{a}$ by $\mathbf{q}$ is $(0, \mathbf{R}(\mathbf{q})\mathbf{a}) = \mathbf{q} \otimes (0, \mathbf{a}) \otimes \mathbf{q}^*$. Expanding with [Eq. \(2.4\)](#) and collecting terms gives [Eq. \(2.6\)](#),

$$\mathbf{R}(\mathbf{q}) = (q_w^2 - \mathbf{q}_v^\top \mathbf{q}_v) \mathbf{I}_3 + 2\mathbf{q}_v \mathbf{q}_v^\top + 2q_w [\mathbf{q}_v]_\times. \quad (\text{A.3})$$

Orthogonality $\mathbf{R}^\top \mathbf{R} = \mathbf{I}$ and $\det \mathbf{R} = +1$ follow for unit $\mathbf{q}$; the conjugate reverses the rotation, $\mathbf{R}(\mathbf{q}^*) = \mathbf{R}(\mathbf{q})^\top$. The composition identity

$$\mathbf{R}(\mathbf{p} \otimes \mathbf{q}) = \mathbf{R}(\mathbf{p}) \mathbf{R}(\mathbf{q}) \quad (\text{A.4})$$

is the statement that $\mathbf{q} \mapsto \mathbf{R}(\mathbf{q})$ is a group homomorphism $\mathbb{S}^3 \rightarrow \text{SO}(3)$; it is two-to-one because $\mathbf{R}(-\mathbf{q}) = \mathbf{R}(\mathbf{q})$. Both [Eq. \(A.4\)](#) and the orthogonality are verified to machine precision in the accompanying `validate_models.py`.

Kinematics

Let $\mathbf{q}(t)$ be the body-to-inertial attitude. A body-frame angular velocity $\boldsymbol{\omega}^b$ over an infinitesimal time dt advances the attitude by the incremental rotation $\text{Exp}(\boldsymbol{\omega}^b dt) \approx (1, \frac{1}{2}\boldsymbol{\omega}^b dt)$, so

$$\mathbf{q}(t + dt) = \mathbf{q}(t) \otimes \left(1, \frac{1}{2}\boldsymbol{\omega}^b dt\right) \Rightarrow \dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes (0, \boldsymbol{\omega}^b), \quad (\text{A.5})$$

which is Eq. (2.8). Writing the right product as a matrix acting on $\mathbf{q}$ gives $\dot{\mathbf{q}} = \frac{1}{2}\mathbf{\Omega}(\boldsymbol{\omega}^b)\mathbf{q}$ with $\mathbf{\Omega}$ of Eq. (2.9). Differentiating $\|\mathbf{q}\|^2 = 1$ gives $\mathbf{q}^\top \dot{\mathbf{q}} = \frac{1}{2}\mathbf{q}^\top \mathbf{\Omega} \mathbf{q} = 0$ (since $\mathbf{\Omega}$ is skew), confirming that the kinematics preserve the unit norm—the property the exponential integrator Eq. (2.11) enforces exactly in discrete time.

Exponential map

For a rotation vector $\boldsymbol{\theta} = \theta \mathbf{u}$, $\|\mathbf{u}\| = 1$, the series $\text{Exp}(\boldsymbol{\theta}) = \sum_n \frac{1}{n!} (0, \frac{1}{2}\boldsymbol{\theta})^{\otimes n}$ sums to Eq. (2.10), $\text{Exp}(\boldsymbol{\theta}) = (\cos \frac{\theta}{2}, \mathbf{u} \sin \frac{\theta}{2})$, the unit quaternion of a rotation by θ about $\mathbf{u}$. Holding $\boldsymbol{\omega}^b$ constant, the exact flow of Eq. (2.8) over Δt is the right product by $\text{Exp}(\boldsymbol{\omega}^b \Delta t)$, giving the norm-exact update Eq. (2.11).

Appendix B

Hexacopter Jacobians

The per-agent solvers of [Chapter 5](#) require the Jacobians of the continuous dynamics [Eq. \(3.12\)](#), which are then propagated through the RK4 stages to the discrete sensitivities [Eq. \(3.13\)](#). With state ordering $\mathbf{x} = (\mathbf{p}, \mathbf{v}, \mathbf{q}, \boldsymbol{\omega})$ and input $\mathbf{u} = (T, \boldsymbol{\tau})$, the continuous Jacobian $\partial \mathbf{f} / \partial \mathbf{x}$ has the block structure

$$\frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{bmatrix} \mathbf{0} & \mathbf{I}_3 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & -\frac{c_D}{m} \mathbf{I}_3 & \frac{T}{m} \frac{\partial(\mathbf{R}(\mathbf{q})\mathbf{e}_3)}{\partial \mathbf{q}} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \frac{1}{2} \boldsymbol{\Omega}(\boldsymbol{\omega}) & \frac{1}{2} \boldsymbol{\Xi}(\mathbf{q}) \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \frac{\partial \dot{\boldsymbol{\omega}}}{\partial \boldsymbol{\omega}} \end{bmatrix}, \quad (\text{B.1})$$

whose non-trivial blocks are as follows.

Thrust-axis block. The third column of $\mathbf{R}(\mathbf{q})$ is $\mathbf{R}(\mathbf{q})\mathbf{e}_3 = 2(q_x q_z + q_w q_y, q_y q_z - q_w q_x, \frac{1}{2} - q_x^2 - q_y^2)^\top$, so

$$\frac{\partial(\mathbf{R}(\mathbf{q})\mathbf{e}_3)}{\partial \mathbf{q}} = 2 \begin{bmatrix} q_y & q_z & q_w & q_x \\ -q_x & -q_w & q_z & q_y \\ 0 & -2q_x & -2q_y & 0 \end{bmatrix}, \quad (\text{B.2})$$

scaled by T/m in the velocity row; this is the linearised attitude-to-translation coupling the controller inverts through differential flatness.

Quaternion-rate blocks. From $\dot{\mathbf{q}} = \frac{1}{2} \boldsymbol{\Omega}(\boldsymbol{\omega}) \mathbf{q} = \frac{1}{2} (0, \boldsymbol{\omega}) \otimes^L \mathbf{q}$, the derivative with respect to $\mathbf{q}$ is $\frac{1}{2} \boldsymbol{\Omega}(\boldsymbol{\omega})$ ([Eq. \(2.9\)](#)), and the derivative with respect to $\boldsymbol{\omega}$ is $\frac{1}{2} \boldsymbol{\Xi}(\mathbf{q})$ with

$$\boldsymbol{\Xi}(\mathbf{q}) = \begin{bmatrix} -q_x & -q_y & -q_z \\ q_w & -q_z & q_y \\ q_z & q_w & -q_x \\ -q_y & q_x & q_w \end{bmatrix} \in \mathbb{R}^{4 \times 3}. \quad (\text{B.3})$$

Angular-rate block. From [Eq. \(3.8\)](#),

$$\frac{\partial \dot{\boldsymbol{\omega}}}{\partial \boldsymbol{\omega}} = \begin{bmatrix} 0 & \frac{J_{yy} - J_{zz}}{J_{xx}} \omega_z & \frac{J_{yy} - J_{zz}}{J_{xx}} \omega_y \\ \frac{J_{zz} - J_{xx}}{J_{yy}} \omega_z & 0 & \frac{J_{zz} - J_{xx}}{J_{yy}} \omega_x \\ \frac{J_{xx} - J_{yy}}{J_{zz}} \omega_y & \frac{J_{xx} - J_{yy}}{J_{zz}} \omega_x & 0 \end{bmatrix}. \quad (\text{B.4})$$

Input Jacobian. The dynamics are affine in the wrench, so

$$\frac{\partial \mathbf{f}}{\partial \mathbf{u}} = \begin{bmatrix} \mathbf{0}_{3 \times 1} & \mathbf{0}_{3 \times 3} \\ \frac{1}{m} \mathbf{R}(\mathbf{q}) \mathbf{e}_3 & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{4 \times 1} & \mathbf{0}_{4 \times 3} \\ \mathbf{0}_{3 \times 1} & \mathbf{J}^{-1} \end{bmatrix}, \quad (\text{B.5})$$

with the thrust column $\frac{1}{m} \mathbf{R}(\mathbf{q}) \mathbf{e}_3$ and the torque block $\mathbf{J}^{-1}$. These closed-form Jacobians were cross-checked against central finite differences in the accompanying code (agreement to $\sim 10^{-6}$, the finite-difference floor).

Appendix C

Vehicle Parameter Tables

The physical parameters used in the models and the simulation study are listed here. The hexacopter values are those of a published multirotor model; the fixed-wing values are the Aerosonde first-edition parameters of Beard and McLain [3].

C.1 Hexacopter

Table C.1: Hexacopter physical parameters (Chapter 3).

Symbol	Meaning	Value	Unit
m	mass	2.468	kg
L	arm length	0.225	m
$J_{xx} = J_{yy}$	roll/pitch inertia	4.856×10^{-3}	kg m^2
J_{zz}	yaw inertia	8.801×10^{-3}	kg m^2
I_r	rotor inertia	3.357×10^{-5}	kg m^2
k_f	thrust coefficient	2.980×10^{-6}	$\text{N s}^2/\text{rad}^2$
k_m	drag-torque coefficient	1.140×10^{-7}	$\text{N m s}^2/\text{rad}^2$
β_i	rotor azimuths	$30^\circ, 90^\circ, \dots, 330^\circ$	–
σ_i	spin directions	$+, -, +, -, +, -$	–

C.2 Fixed-wing (Aerosonde)

The trim of Chapter 4 and the fixed-wing simulation of Chapter 10 use these values; the numerically computed straight-and-level trim at 35 m/s is $(\alpha^*, \delta_e^*, \delta_t^*) = (0.20^\circ, -2.83^\circ, 0.464)$.

Table C.2: Aerosonde airframe, propulsion and geometric parameters (Chapter 4).

Symbol	Meaning	Value	Unit
m	mass	13.5	kg
J_x, J_y, J_z	inertias	0.8244, 1.135, 1.759	kg m ²
J_{xz}	product of inertia	0.1204	kg m ²
S	wing area	0.55	m ²
b	wingspan	2.8956	m
c	mean chord	0.18994	m
S_{prop}	propeller area	0.2027	m ²
ρ	air density	1.2682	kg/m ³
e	Oswald efficiency	0.9	–
k_{motor}	motor constant	80	–
C_{prop}	propeller efficiency	1.0	–
M, α_0	stall sharpness, angle	50, 0.4712	–, rad
ϵ	downwash	0.1592	–

Table C.3: Aerosonde aerodynamic coefficients. Longitudinal (top), lateral (bottom).

Longitudinal					
C_{L_0}	0.28	C_{D_0}	0.03	C_{m_0}	–0.02338
C_{L_α}	3.45	C_{D_α}	0.30	C_{m_α}	–0.38
C_{L_q}	0.0	C_{D_p}	0.0437	C_{m_q}	–3.6
$C_{L_{\delta_e}}$	–0.36	$C_{D_{\delta_e}}$	0.0	$C_{m_{\delta_e}}$	–0.5
Lateral					
C_{Y_0}	0.0	C_{ℓ_0}	0.0	C_{n_0}	0.0
C_{Y_β}	–0.98	C_{ℓ_β}	–0.12	C_{n_β}	0.25
C_{Y_p}	0.0	C_{ℓ_p}	–0.26	C_{n_p}	0.022
C_{Y_r}	0.0	C_{ℓ_r}	0.14	C_{n_r}	–0.35
$C_{Y_{\delta_a}}$	0.0	$C_{\ell_{\delta_a}}$	0.08	$C_{n_{\delta_a}}$	0.06
$C_{Y_{\delta_r}}$	–0.17	$C_{\ell_{\delta_r}}$	0.105	$C_{n_{\delta_r}}$	–0.032

Bibliography

- [1] G. Michieletto, M. Ryll, and A. Franchi, “Fundamental actuation properties of multirotors: Force–moment decoupling and fail–safe robustness,” *IEEE Transactions on Robotics*, vol. 34, no. 3, pp. 702–715, 2018.
- [2] D. Mellinger and V. Kumar, “Minimum snap trajectory generation and control for quadrotors,” in *IEEE Int. Conf. on Robotics and Automation (ICRA)*, pp. 2520–2525, 2011.
- [3] R. W. Beard and T. W. McLain, *Small Unmanned Aircraft: Theory and Practice*. Princeton University Press, 2012.
- [4] M. Diehl, H. G. Bock, and J. P. Schlöder, “A real-time iteration scheme for nonlinear optimization in optimal feedback control,” *SIAM Journal on Control and Optimization*, vol. 43, no. 5, pp. 1714–1736, 2005.
- [5] S. Gros, M. Zanon, R. Quirynen, A. Bemporad, and M. Diehl, “From linear to nonlinear MPC: bridging the gap via the real-time iteration,” *International Journal of Control*, vol. 93, no. 1, pp. 62–80, 2020.
- [6] C. V. Rao, J. B. Rawlings, and D. Q. Mayne, “Constrained state estimation for nonlinear discrete-time systems: stability and moving horizon approximations,” *IEEE Transactions on Automatic Control*, vol. 48, no. 2, pp. 246–258, 2003.
- [7] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2nd ed., 2017.
- [8] M. Farina, G. Ferrari-Trecate, and R. Scattolini, “Distributed moving horizon estimation for linear constrained systems,” *IEEE Transactions on Automatic Control*, vol. 55, no. 11, pp. 2462–2475, 2010.
- [9] M. Farina, G. Ferrari-Trecate, and R. Scattolini, “Distributed moving horizon estimation for nonlinear constrained systems,” *International Journal of Robust and Nonlinear Control*, vol. 22, no. 2, pp. 123–143, 2012.
- [10] G. Battistelli, “Distributed moving-horizon estimation with arrival-cost consensus,” *IEEE Transactions on Automatic Control*, vol. 64, no. 8, pp. 3316–3323, 2019.
- [11] R. Schneider and W. Marquardt, “Convergence and stability of a constrained partition-based moving horizon estimator,” *IEEE Transactions on Automatic Control*, vol. 61, no. 5, pp. 1316–1321, 2016.
- [12] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, “Distributed optimization and statistical learning via the alternating direction method of multipliers,” *Foundations and Trends in Machine Learning*, vol. 3, no. 1, pp. 1–122, 2011.

- [13] T. H. Summers and J. Lygeros, “Distributed model predictive consensus via the alternating direction method of multipliers,” in *50th Allerton Conf. on Communication, Control, and Computing*, pp. 79–84, 2012.
- [14] P. Giselsson, M. D. Doan, T. Keviczky, B. De Schutter, and A. Rantzer, “Accelerated gradient methods and dual decomposition in distributed model predictive control,” *Automatica*, vol. 49, no. 3, pp. 829–833, 2013.
- [15] B. T. Stewart, A. N. Venkat, J. B. Rawlings, S. J. Wright, and G. Pannocchia, “Cooperative distributed model predictive control,” *Systems & Control Letters*, vol. 59, no. 8, pp. 460–469, 2010.
- [16] C. Conte, C. N. Jones, M. Morari, and M. N. Zeilinger, “Distributed synthesis and stability of cooperative distributed model predictive control for linear systems,” *Automatica*, vol. 69, pp. 117–125, 2016.
- [17] P. D. Christofides, R. Scattolini, D. Muñoz de la Peña, and J. Liu, “Distributed model predictive control: A tutorial review and future research directions,” *Computers & Chemical Engineering*, vol. 51, pp. 21–41, 2013.
- [18] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, “Control barrier function based quadratic programs for safety critical systems,” *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017.
- [19] L. Wang, A. D. Ames, and M. Egerstedt, “Safety barrier certificates for collisions-free multi-robot systems,” *IEEE Transactions on Robotics*, vol. 33, no. 3, pp. 661–674, 2017.
- [20] K.-K. Oh, M.-C. Park, and H.-S. Ahn, “A survey of multi-agent formation control,” *Automatica*, vol. 53, pp. 424–440, 2015.
- [21] W. Ren and R. W. Beard, “Formation feedback control for multiple spacecraft via virtual structures,” *IEE Proceedings – Control Theory and Applications*, vol. 151, no. 3, pp. 357–368, 2004.
- [22] L. Krick, M. E. Broucke, and B. A. Francis, “Stabilisation of infinitesimally rigid formations of multi-robot networks,” *International Journal of Control*, vol. 82, no. 3, pp. 423–439, 2009.
- [23] R. Olfati-Saber and R. M. Murray, “Consensus problems in networks of agents with switching topology and time-delays,” *IEEE Transactions on Automatic Control*, vol. 49, no. 9, pp. 1520–1533, 2004.
- [24] R. Olfati-Saber, J. A. Fax, and R. M. Murray, “Consensus and cooperation in networked multi-agent systems,” *Proceedings of the IEEE*, vol. 95, no. 1, pp. 215–233, 2007.
- [25] J. A. Fax and R. M. Murray, “Information flow and cooperative control of vehicle formations,” *IEEE Transactions on Automatic Control*, vol. 49, no. 9, pp. 1465–1476, 2004.